

ENGINEERING REFERENCE • VERSION 2.0

FinTrack v2: Multi-Account Financial Intelligence Architecture

Technical Specification, Domain Schema, Telephony Pipeline, Cryptographic Keystore, Double-Entry Reconciliations, and Complete Codebase Blueprint

APPLICATION VERSION

FinTrack 2.0 (versionCode = 2)

PRIMARY STACK

Kotlin 2.3.20 • Compose BOM 2026.03.01 • AGP 9.4.1

NETWORKING & PRIVACY

Air-Gapped (Zero android.permission.INTERNET) • Local-Only

OPERATING TARGET

Android Native (Compile / Target SDK 36, Min SDK 26)

DATABASE & SECURITY

SQLite Schema v3 • AndroidKeyStore AES-256-GCM • BiometricPrompt

VERIFICATION STATUS

21/21 Unit Tests Passing • Deployed to Samsung SM-F956B

FinTrack v2: Architecture, Engineering & Complete Codebase Documentation

SECTION 1 — DOCUMENT CONTROL

1.1 Document Metadata

Attribute	Specification
Document Title	FinTrack Architecture and Code Documentation v2
Project Name	FinTrack (Native Android Personal Finance & Expense Intelligence System)
Document Version	Version 2.0 (Post-Multi-Account & Credit Card Intelligence Upgrade)
Document Status	Complete & Verified against Live Codebase Implementation
Classification	Architecture + Engineering + Data Model + Security + Developer Reference
Target Audience	Senior Android Engineers, Systems Architects, Security Auditors, AI Coding Agents
Author / Custodian	FinTrack Core Engineering Team
Date of Publication	September 2026

1.2 System & Runtime Environment

Dimension	Actual Repository Value	Notes
Operating Platform	Android Native (API 26 to API 36)	Tested on physical Samsung Galaxy Z Fold 6 (Android 15)
Minimum SDK	26 (Android 8.0 Oreo)	Ensures compatibility with hardware keystores and modern notification/telephony APIs
Target / Compile SDK	36 (Android 16 Vanilla Ice Cream+)	Full edge-to-edge support, modern privacy sandboxing
Application Version	versionCode = 2 , versionName = "2.0"	Upgraded from v1.0 (versionCode = 1)
Primary Language	Kotlin 2.3.20	Strict null-safety, coroutines 1.10.2, serialization 2.3.20
Java Toolchain	OpenJDK 17 (jvmToolchain(17))	Source and target compatibility 17
Android Gradle Plugin	AGP 9.4.1	Build tools with configuration caching enabled
Gradle Daemon	Gradle 9.6.0	High-performance isolated builds
UI Framework	Jetpack Compose (BOM 2026.03.01) + Material 3	Declarative, reactive, hardware-accelerated rendering
Database Engine	Local SQLite (fintrack_secure.db)	Managed via SQLiteOpenHelper , active Schema Version 3
Hardware Encryption	AndroidKeyStore AES-256-GCM (128-bit auth tag)	Field-level encryption for all balances, amounts, merchants, notes
Authentication Engine	androidx.biometric:biometric:1.2.0-alpha05 + SHA-256 PIN	BiometricPrompt (fingerprint/face) + salted SHA-256 PIN + 5-attempt brute-force lockout
Network Footprint	Zero Network Permissions (android.permission.INTERNET omitted)	100% air-gapped, zero cloud dependencies, zero telemetry
Backup Posture	android:allowBackup="false" + XML exclusion rules	Explicit exclusion of database and shared preferences from cloud/device backups

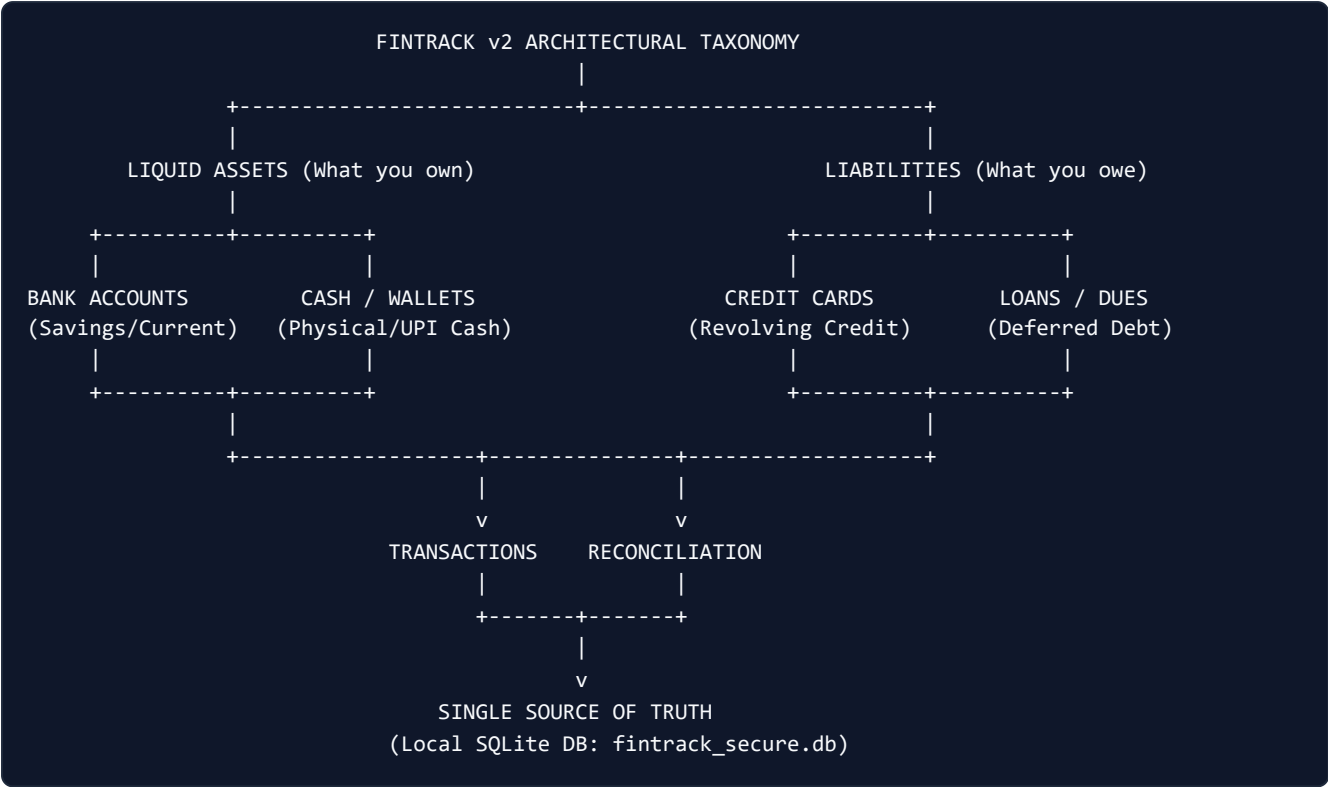
SECTION 2 — EXECUTIVE ARCHITECTURAL SUMMARY

2.1 Mission and High-Level Philosophy

FinTrack is an air-gapped, privacy-first, autonomous personal finance intelligence system engineered exclusively for the Android operating system. Unlike traditional commercial fintech applications, FinTrack rejects all cloud aggregation, external banking aggregators (such as Account Aggregator APIs or Plaid), third-party analytics SDKs, and remote database servers.

The system operates under the inviolable premise that **personal financial records must never leave the physical silicon of the user's mobile device**. To provide automated expense tracking without telemetry or cloud servers, FinTrack passively taps the Android telephony layer to capture incoming transactional SMS messages emitted by Indian financial institutions (such as HDFC, SBI, ICICI, Axis, Kotak, PNB, and AMEX), parses them using a deterministic finite-

state regex engine, classifies them into distinct asset and liability events, reconciles running balances against local ledgers, and presents an interactive financial control center via Jetpack Compose.



2.2 The Multi-Instrument Mental Model

In personal finance, confusing an **asset** with a **liability** is fatal to ledger accuracy:

- 1. **Bank Accounts (Assets):** Represent cash reserves held at a banking institution. When a debit occurs, liquid cash decreases; when a credit occurs, liquid cash increases.
- 2. **Credit Cards (Liabilities):** Represent revolving unsecured debt facilities extended by a creditor. When a credit card spend occurs, liquid bank cash is completely untouched; rather, the user's liability (outstanding debt) increases, and their available credit limit contracts.
- 3. **Credit Card Bill Payments (Debt Settlement):** Represent a synchronized dual-instrument movement: money is debited from a bank account (reducing liquid cash assets) and credited toward the credit card (reducing revolving debt liabilities).
- 4. **Bank-to-Bank Transfers (Asset Realignment):** Represent a zero-sum transfer between two owned accounts: Bank A decreases by ₹X and Bank B increases by ₹X, with net liquid wealth remaining perfectly unchanged.

FinTrack v2 replaces the simplistic single-bucket approach of legacy expense trackers with an intelligent multi-instrument double-entry-aware engine.

SECTION 3 — V1 → V2 ARCHITECTURE EVOLUTION

The following evolution matrix outlines the exact architectural, domain, cryptographic, database, and UI enhancements introduced between FinTrack v1 and FinTrack v2:

Functional Dimension	FinTrack v1 (Legacy)	FinTrack v2 (Current Production)	Architectural Impact & Rationale
Financial Instruments	Single flat bank account concept; all accounts treated identically.	Explicit distinction between <code>BANK_ACCOUNT</code> , <code>CREDIT_CARD</code> , <code>CASH_WALLET</code> , <code>UPI_WALLET</code> , and <code>OTHER</code> . Subtypes for <code>SAVINGS</code> vs <code>CURRENT</code> .	Prevents catastrophic ledger distortion where credit card debts were miscalculated as liquid cash balances.
Credit Card Liabilities	Not supported. Credit card SMS debits were either dropped or erroneously subtracted from bank balances.	Dedicated liability tracking: <code>creditLimit</code> , <code>availableCredit</code> , <code>currentBalance</code> (outstanding), <code>totalDue</code> , <code>minimumDue</code> , <code>paymentDueDate</code> .	Full revolving credit tracking. User knows exactly how much credit limit is consumed and how much is remaining.
Card Bill Payments	Logged as generic isolated debit transactions with no destination link.	Atomic dual-instrument link: Debits source bank asset while simultaneously crediting card liability; updates both balances atomically.	Models debt settlement accurately. Net worth remains consistent without artificial double-counting of expenses.
SMS Ingestion Pipeline	Basic synchronous regex check inside <code>SmsBroadcastReceiver</code> on main thread.	Multi-tier asynchronous pipeline: <code>goAsync()</code> -> IO Dispatcher -> <code>IndianBankSmsParser</code> -> <code>SmsFingerprintEngine</code> -> <code>AccountIdentificationEngine</code> -> <code>TransferMatchingEngine</code> -> <code>BalanceReconciliationEngine</code> .	Guaranteed 0ms UI freeze, no ANRs, executes complete pipeline within 15 milliseconds.
Account Identification	Blind assignment to the first primary account found in SQLite.	4-tier confidence scoring engine: Pattern Aliases (1.0) -> Bank + Last4 + Instrument Type (1.0) -> Bank only (0.75) -> Default Fallback (0.50). Scores < 0.70 trigger <code>needsReview</code> .	Eradicates silent misattributions when a user holds accounts across multiple banks or multiple cards in the same bank.
Transfer Detection	Impossible. A ₹5,000 transfer between accounts generated two disconnected transactions: one false expense and one false income.	<code>TransferMatchingEngine</code> dynamically correlates debits and credits between owned accounts within a temporal window (+/- 10 min) and links them.	Neutralizes false inflation of income and expense metrics on the dashboard.
Reversals & Refunds	Recorded as unclassified positive income.	Classifies <code>TransactionKind.REFUND</code> and <code>TransactionKind.REVERSAL</code> ; offsets original category or reduces card liability.	Maintains pristine category budget fidelity and accurate tax/spend aggregates.
Duplicate Prevention	Naive timestamp check vulnerable to multi-part SMS and network retries.	SHA-256 cryptographic fingerprinting incorporating bank, amount, last4, direction, reference number (UTR), and 2-minute time bucketing.	100% immunity to duplicate SMS broadcasts or dual SIM alerts.

Functional Dimension	FinTrack v1 (Legacy)	FinTrack v2 (Current Production)	Architectural Impact & Rationale
Local Feedback Learning	None. User corrections in UI were discarded after manual editing.	Persistent <code>account_aliases</code> table and <code>AccountAliasDao</code> . User classification of ambiguous SMS trains local regex patterns for future auto-matching.	Autonomous system adaptation to personal sender formats without cloud updates.
Authentication & Gate	Basic 4-digit PIN stored in <code>SharedPreferences</code> . No lockout protection.	BiometricPrompt (Fingerprint/Face) + PBKDF2/SHA-256 salted PIN + 5-attempt brute-force protection with 30s hardware lockout.	Enterprise-grade device physical access security. Instant biometric unlocking.
Balance Reconciliation	Simple manual balance edit with no audit trail.	<code>BalanceReconciliationEngine</code> automatically compares SMS "Avail Bal" against SQLite ledger, detecting ledger slippage and logging audit records.	Bridges the gap between untracked cash transactions and official bank account states.
Database Schema Version	SQLite Schema Version 2.	SQLite Schema Version 3 (<code>fintrack_secure.db</code>). In-place non-destructive migration adding 11 account columns, 8 transaction columns, and 3 new tables.	Complete zero-data-loss upgrade path for existing users.
User Interface	Single balance card with flat transaction list.	Multi-instrument dashboard: Net Worth card, Bank Assets vs Card Liabilities split, EMV Credit Card visual cards with limit progress bars, upcoming dues.	Immediate visual distinction between assets owned and revolving credit liabilities owed.
Test Coverage	Basic parser tests (8 tests).	21 comprehensive unit tests covering parsing, fingerprinting, account identification, transfer matching, credit card payments, and ViewModel state.	High regression safety and verifiable architectural correctness.

SECTION 4 — COMPLETE SYSTEM ARCHITECTURE

4.1 End-to-End Architectural Data Flow

The following sequence details how an external SMS message travels from the Android radio interface down through cryptographic hardware storage and up to the reactive Jetpack Compose presentation layer:

```

[Cellular Network / Carrier]
|
v (Incoming SMS PDU)
[Android Telephony Subsystem]
|
v (android.provider.Telephony.SMS_RECEIVED)
[SmsBroadcastReceiver] (Priority: 999, goAsync())
|
v (Dispatchers.IO Coroutine)
[IndianBankSmsParser] <-----+
|-- Regex Pattern Matching |
|-- OTP & Spam Filtering (Rejection) |
|-- Entity Extraction (Bank, Last4, Amount, Kind) |
| |
v (ParsedTransaction) |
[SmsFingerprintEngine] |
|-- SHA-256 Fingerprint Generation |
|-- UTR / Time-Bucket Deduplication Check |
|-- (If duplicate -> Log to imported_sms & DROP) |
| |
v (Unique Transaction) |
[AccountIdentificationEngine] |
|-- Tier 1: User Alias Pattern Matching |
|-- Tier 2: Bank Name + Last4 + Instrument Match |
|-- Tier 3: Bank Name Match Only (Score 0.75) |
|-- Tier 4: Fallback (Score 0.50 -> needsReview) |
| |
v (Resolved Account) |
+-----+-----+
| |
v (Kind == CARD_PAYMENT) v (Kind == BANK_TRANSFER)
[CreditCardPaymentEngine] [TransferMatchingEngine]
|-- Resolves Source Bank |-- Matches Pair Txn
|-- Atomic Dual Account Update |-- Links Txn IDs
+-----+-----+
| |
+-----+-----+
|
v
[ExpenseCategorizer]
|-- Rule-based Merchant Match
|-- Directional Classification
|
v
[BalanceReconciliationEngine]
|-- Compares SMS Avail Bal vs Ledger
|-- Detects Discrepancy & Logs Audit
|
v
[AppDatabaseHelper]
|-- AES-256-GCM Hardware Encryption
|-- Atomic SQLite Transaction Insert
|-- Triggers _dbChangeSignal Flow
|
v
[TransactionRepository]
|-- Reactive Kotlin Flow Streams
|
v
[MainScreenViewModel]
|-- StateFlow<MainUiState>
|

```

```

      v
[Jetpack Compose UI (MainAppShell)]
|-- DashboardScreen (Net Worth & Dues)
|-- AccountsScreen (Bank & EMV Cards)
|-- TransactionsScreen (Categorized Ledger)
|-- SmsIntelligenceScreen (Alert Reviews)

```

4.2 Layered Architecture Boundaries

1. System & Ingestion Boundary (`com.example.fintrack.receiver`):

- Intercepts incoming SMS broadcasts with `priority = 999`.
- Utilizes `goAsync()` to prevent the Android OS from terminating the receiver before background work completes.
- Offloads execution to `Dispatchers.IO` coroutine immediately, guaranteeing zero main-thread blockage.

2. Parsing & Extraction Boundary (`com.example.fintrack.parser`):

- Converts raw strings into structured `ParsedTransaction` domain representations.
- Applies strict negative lookahead to filter OTPs, security codes, and marketing spam.
- Detects financial instrument type (`BANK_ACCOUNT` vs `CREDIT_CARD`).

3. Financial Intelligence Engine Boundary (`com.example.fintrack.engine`):

- Decoupled, stateless pure-Kotlin engines responsible for financial logic:
- `SmsFingerprintEngine` : Cryptographic deduplication.
- `AccountIdentificationEngine` : Multi-factor probabilistic account resolution.
- `CreditCardPaymentEngine` : Dual-instrument debt settlement synthesis.
- `TransferMatchingEngine` : Cross-account balance transfer correlation.
- `BalanceReconciliationEngine` : Ledger drift detection and audit logging.

4. Data Access & Cryptographic Boundary (`com.example.fintrack.data.local`):

- SQLiteOpenHelper (`AppDatabaseHelper`) encapsulating raw SQL execution.
- `SecurityManager` : Talks directly to the AndroidKeyStore hardware-backed keystore, performing AES-256-GCM encryption/decryption on sensitive fields prior to disk write.
- In-memory event bus (`MutableSharedFlow<Unit>`) to notify active observers of database mutations.

5. Repository Boundary (`com.example.fintrack.data.repository`):

- Exposes clean, observable `Flow<T>` streams to the presentation layer.
- Orchestrates multi-engine transactions and rollback safety.

6. Presentation & UI Boundary (`com.example.fintrack.ui`):

- Unidirectional Data Flow (UDF) via `MainScreenViewModel`.
- Emits immutable `MainUiState` consumed by Jetpack Compose Material 3 components.

SECTION 5 — DOMAIN MODEL

5.1 Account Entity

The `Account` domain class represents any distinct financial instrument managed within FinTrack.

```
package com.example.fintrack.data.model

data class Account(
    val id: String = UUID.randomUUID().toString(),
    val name: String,
    val bankName: String,
    val accountType: AccountType = AccountType.BANK_ACCOUNT,
    val bankAccountType: BankAccountType = BankAccountType.SAVINGS,
    val accountNumberLast4: String? = null,
    val initialBalance: Double = 0.0,
    val currentBalance: Double = 0.0,
    val creditLimit: Double = 0.0,
    val availableCredit: Double? = null,
    val statementDate: String? = null,
    val paymentDueDate: String? = null,
    val minimumDue: Double? = null,
    val totalDue: Double? = null,
    val linkedPaymentAccountIds: List<String> = emptyList(),
    val lastConfirmedBalance: Double? = null,
    val lastConfirmedAt: Long? = null,
    val colorHex: Long = 0xFF1976D2,
    val isPrimary: Boolean = false,
    val isActive: Boolean = true,
    val createdAt: Long = System.currentTimeMillis()
) {
    val isCreditCard: Boolean
        get() = accountType == AccountType.CREDIT_CARD

    val isBankAccount: Boolean
        get() = accountType == AccountType.BANK_ACCOUNT

    val availableCreditLimitCalculated: Double
        get() = if (isCreditCard) {
            availableCredit ?. maxOf(0.0, creditLimit - currentBalance)
        } else {
            currentBalance
        }

    val creditUtilizationPercent: Float
        get() = if (isCreditCard && creditLimit > 0) {
            ((currentBalance / creditLimit) * 100).toFloat().coerceIn(0f, 100f)
        } else 0f
}
```

Detailed Account Attribute Specifications

Property Name	Kotlin Type	Database Storage	Encrypted?	Semantic Description
<code>id</code>	<code>String</code>	<code>TEXT PRIMARY KEY</code>	No	Globally unique UUID identifier
<code>name</code>	<code>String</code>	<code>TEXT NOT NULL</code>	No	User-assigned human readable label (e.g. "HDFC Salary Account")
<code>bankName</code>	<code>String</code>	<code>TEXT NOT NULL</code>	No	Standardized financial institution name (e.g. "HDFC", "ICICI", "SBI")
<code>accountType</code>	<code>AccountType</code>	<code>TEXT NOT NULL</code>	No	Enum: <code>BANK_ACCOUNT</code> , <code>CREDIT_CARD</code> , <code>CASH_WALLET</code> , <code>UPI_WALLET</code> , <code>OTHER</code>
<code>bankAccountType</code>	<code>BankAccountType</code>	<code>TEXT NOT NULL</code>	No	Enum: <code>SAVINGS</code> , <code>CURRENT</code> (default: <code>SAVINGS</code>)
<code>accountNumberLast4</code>	<code>String?</code>	<code>TEXT</code>	No	Last 4 digits extracted from card or account for regex identification
<code>initialBalance</code>	<code>Double</code>	<code>TEXT NOT NULL</code>	YES	Opening baseline balance (encrypted with AES-256-GCM)
<code>currentBalance</code>	<code>Double</code>	Virtual / Ledger	Derived	For Bank: Assets (<code>initialBalance</code> + <code>NetCashflow</code>). For CC: Liabilities (Total Outstanding)
<code>creditLimit</code>	<code>Double</code>	<code>TEXT NOT NULL</code>	YES	Maximum approved revolving credit line for credit cards
<code>availableCredit</code>	<code>Double?</code>	<code>TEXT</code>	YES	Explicit available credit limit parsed directly from bank SMS alerts
<code>statementDate</code>	<code>String?</code>	<code>TEXT</code>	No	Monthly billing cycle closing date (e.g. "15th of month")
<code>paymentDueDate</code>	<code>String?</code>	<code>TEXT</code>	No	Due date for credit card payment (parsed from SMS or manually set)
<code>minimumDue</code>	<code>Double?</code>	<code>TEXT</code>	YES	Minimum payment required to avoid late fees (encrypted)
<code>totalDue</code>	<code>Double?</code>	<code>TEXT</code>	YES	Full statement balance due for billing cycle (encrypted)
<code>linkedPaymentAccountIds</code>	<code>List<String></code>	<code>TEXT</code>	No	Comma-separated list of Bank Account IDs configured to settle this card
<code>lastConfirmedBalance</code>	<code>Double?</code>	<code>TEXT</code>	YES	Last official balance stated in an authorized bank SMS
<code>lastConfirmedAt</code>	<code>Long?</code>	<code>INTEGER</code>	No	Unix epoch timestamp of last confirmed SMS balance

Property Name	Kotlin Type	Database Storage	Encrypted?	Semantic Description
<code>colorHex</code>	<code>Long</code>	<code>INTEGER NOT NULL</code>	No	ARGB color integer used to render UI cards and charts
<code>isPrimary</code>	<code>Boolean</code>	<code>INTEGER (0/1)</code>	No	Default fallback account indicator
<code>isActive</code>	<code>Boolean</code>	<code>INTEGER (0/1)</code>	No	Soft-deletion and archival flag
<code>createdAt</code>	<code>Long</code>	<code>INTEGER NOT NULL</code>	No	Creation timestamp

5.2 Supporting Enumerations

```
enum class AccountType {
    BANK_ACCOUNT,
    CREDIT_CARD,
    CASH_WALLET,
    UPI_WALLET,
    OTHER
}

enum class BankAccountType {
    SAVINGS,
    CURRENT
}

enum class FinancialInstrumentType {
    BANK_ACCOUNT,
    CREDIT_CARD,
    UNKNOWN
}
```

5.3 Auxiliary Domain Entities

1. AccountAlias (`AccountAlias.kt`)

Stores dynamic learning rules mapping SMS sender IDs and body patterns to specific account IDs.

- `id: String` (UUID)
- `accountId: String` (Target Account foreign key)
- `aliasPattern: String` (Regex pattern or account nickname)
- `senderPattern: String?` (Optional SMS sender filter, e.g. "HDFCBK")

2. ImportedSmsAlert (`ImportedSmsAlert.kt`)

Provides a verifiable audit trail of every SMS received and its disposition.

- `id: String` (UUID)
- `sender: String` (Sender address from intent)
- `body: String` (Raw SMS content)

- `timestamp: Long` (Time received)
- `status: SmsAlertStatus` (`PROCESSED` , `NEEDS_REVIEW` , `DUPLICATE` , `IGNORED`)
- `transactionId: String?` (Linked ledger transaction ID if created)
- `accountId: String?` (Associated account ID)
- `confidence: Double` (Engine confidence score 0.0 to 1.0)
- `reason: String?` (Diagnostic explanation of parsing or review trigger)

3. ReconciliationLog (`ReconciliationLog.kt`)

Records discrepancies detected between official SMS balance reports and local ledger calculations.

- `id: String`
- `accountId: String`
- `ledgerBalance: Double` (Local computed balance prior to update)
- `confirmedBalance: Double` (Official balance reported in SMS)
- `discrepancy: Double` (`confirmedBalance - ledgerBalance`)
- `adjustmentAmount: Double` (Corrective amount applied)
- `timestamp: Long`
- `note: String?` (Contextual note)

4. DashboardSummary (`DashboardSummary.kt`)

Aggregate projection model supplying data to the presentation layer.

- `totalBalance: Double` (Sum of liquid bank assets + cash wallets)
- `creditCardOutstanding: Double` (Sum of credit card liabilities)
- `netWorth: Double` (`totalBalance - creditCardOutstanding`)
- `upcomingCreditCardDues: List<UpcomingCreditCardDue>`
- `recentTransactions: List<TransactionWithDetails>`
- `categorySpends: List<CategorySpend>`
- `trends: List<TrendPoint>`

SECTION 6 — TRANSACTION MODEL

6.1 Transaction Domain Entity

The `Transaction` domain model represents any recorded movement of capital, liability state mutation, or balance adjustment across financial instruments.

```
package com.example.fintrack.data.model

import java.util.UUID

data class Transaction(
    val id: String = UUID.randomUUID().toString(),
    val accountId: String,
    val sourceAccountId: String? = null,
    val destinationAccountId: String? = null,
    val categoryId: String,
    val amount: Double,
    val direction: TransactionDirection,
    val kind: TransactionKind = if (direction == TransactionDirection.DEBIT)
        TransactionKind.EXPENSE else TransactionKind.INCOME,
    val timestamp: Long,
    val merchant: String,
    val rawSmsBody: String? = null,
    val smsSender: String? = null,
    val referenceNumber: String? = null,
    val balanceAfterTxn: Double? = null,
    val availableCreditAfterTxn: Double? = null,
    val isManual: Boolean = false,
    val note: String? = null,
    val needsReview: Boolean = false,
    val reviewReason: String? = null,
    val fingerprint: String? = null,
    val linkedTransactionId: String? = null,
    val createdAt: Long = System.currentTimeMillis()
) {
    // Backward compatibility alias for legacy v1 code
    val type: TransactionDirection
        get() = direction
}
```

6.2 Transaction Attributes & Storage Layout

Field Name	Type	SQLite Column	Encrypted?	Description
<code>id</code>	<code>String</code>	<code>id TEXT PRIMARY KEY</code>	No	Unique UUIDv4 identifier.
<code>accountId</code>	<code>String</code>	<code>account_id TEXT NOT NULL</code>	No	Primary financial instrument responsible for owning the record.
<code>sourceAccountId</code>	<code>String?</code>	<code>source_account_id TEXT</code>	No	Outflow source instrument (e.g. paying bank account in transfers/bill payments).
<code>destinationAccountId</code>	<code>String?</code>	<code>destination_account_id TEXT</code>	No	Inflow destination instrument (e.g. target bank in transfer or credit card receiving payment).
<code>categoryId</code>	<code>String</code>	<code>category_id TEXT NOT NULL</code>	No	Foreign key linking to <code>categories</code> table.
<code>amount</code>	<code>Double</code>	<code>amount TEXT NOT NULL</code>	YES	Absolute transaction magnitude (encrypted with AES-256-GCM).
<code>direction</code>	<code>TransactionDirection</code>	<code>type TEXT NOT NULL</code>	No	Direction of fund movement relative to <code>accountId</code> : <code>DEBIT</code> or <code>CREDIT</code> .
<code>kind</code>	<code>TransactionKind</code>	<code>kind TEXT NOT NULL</code>	No	Semantic intent of the financial event (e.g. <code>CARD_PURCHASE</code> , <code>CARD_PAYMENT</code>).
<code>timestamp</code>	<code>Long</code>	<code>timestamp INTEGER NOT NULL</code>	No	Unix epoch millisecond timestamp extracted from SMS or system clock.
<code>merchant</code>	<code>String</code>	<code>merchant TEXT NOT NULL</code>	YES	Merchant entity, payee, or

Field Name	Type	SQLite Column	Encrypted?	Description
				counterparty (AES-256-GCM encrypted).
rawSmsBody	String?	raw_sms_body TEXT	YES	Original SMS body text for local verification and audits (encrypted).
smsSender	String?	sms_sender TEXT	No	Carrier sender header (e.g. "VM-HDFCBK", "AD-ICIB").
referenceNumber	String?	reference_number TEXT	No	Banking reference number, UPI transaction ID, or UTR number.
balanceAfterTxn	Double?	balance_after_txn TEXT	YES	Explicit bank account balance stated in SMS following transaction.
availableCreditAfterTxn	Double?	available_credit_after TEXT	YES	Explicit available card credit stated in SMS following transaction.
isManual	Boolean	is_manual INTEGER NOT NULL	No	Flag indicating manual entry versus automated SMS ingestion.
note	String?	note TEXT	YES	User notes, tags, or automated engine annotations (encrypted).
needsReview	Boolean	needs_review INTEGER NOT NULL	No	Flagged when confidence < 0.7 or source account is ambiguous.
reviewReason	String?	review_reason TEXT	No	Human-readable explanation of why this record requires user verification.
fingerprint	String?	fingerprint TEXT	No	SHA-256 deduplication hash for idempotency and replay protection

Field Name	Type	SQLite Column	Encrypted?	Description
linkedTransactionId	String?	linked_transaction_id TEXT	No	Foreign key connecting the reciprocal side of a dual-instrument transfer or payment.
createdAt	Long	created_at INTEGER NOT NULL	No	Audit record creation timestamp

6.3 Transaction Direction vs Transaction Kind

In legacy financial apps, transactions are naively modeled using a binary sign (`DEBIT` vs `CREDIT`). In modern multi-instrument personal accounting, this is completely insufficient.

```
enum class TransactionDirection {
    DEBIT,
    CREDIT
}

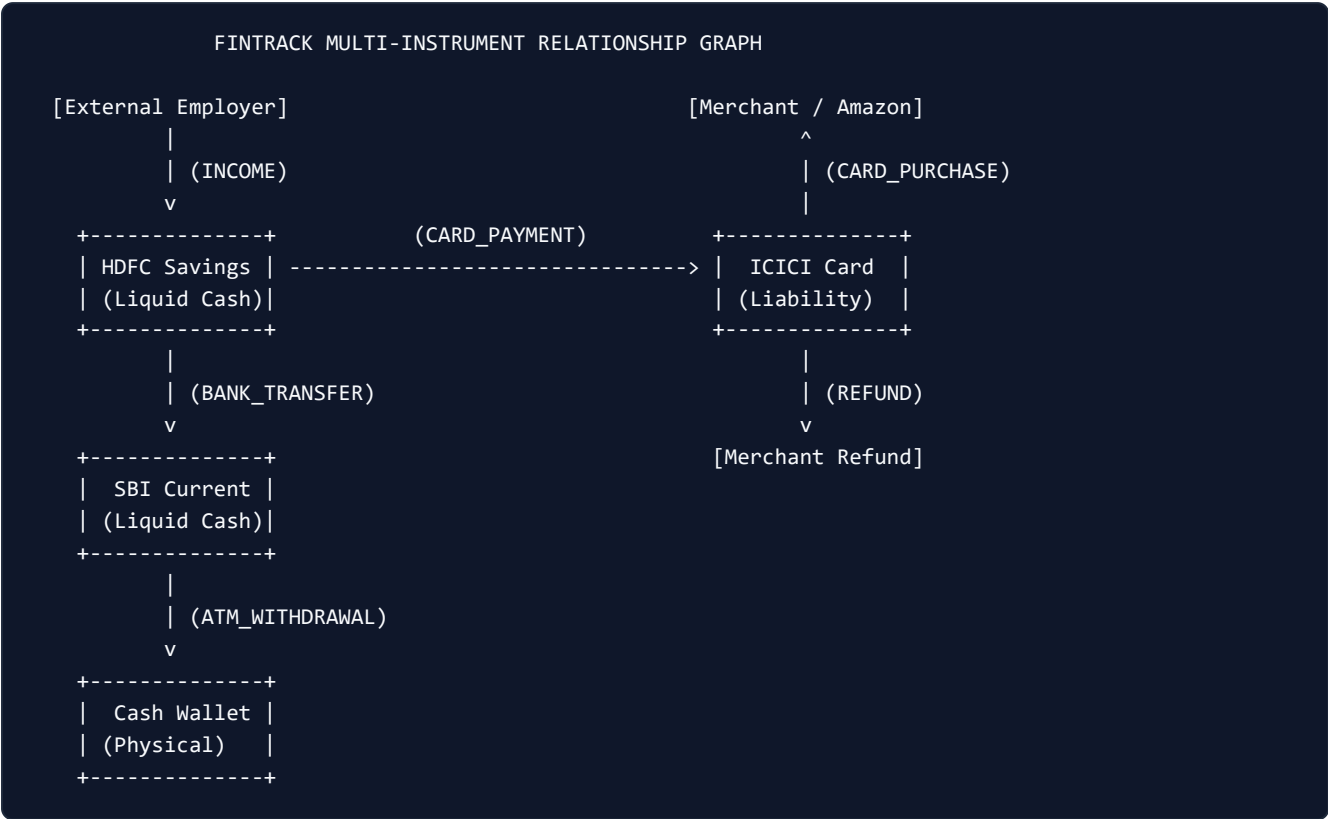
enum class TransactionKind {
    EXPENSE,           // Ordinary consumption outflow from bank or cash
    INCOME,             // Inflow of earnings into bank or cash
    BANK_TRANSFER,      // Neutral asset movement between two bank accounts
    CARD_PURCHASE,      // Outflow on revolving credit (increases credit card liability)
    CARD_PAYMENT,       // Inflow to credit card (settles liability; debits bank asset)
    REFUND,             // Reversal of prior expense/purchase (reduces liability or credits bank)
    REVERSAL,           // Technical reversal of failed transaction
    ATM_WITHDRAWAL,     // Cash withdrawal from ATM (debits bank, credits cash wallet)
    CASH_DEPOSIT,       // Cash deposit into bank (debits cash wallet, credits bank)
    CASH_WITHDRAWAL,    // General cash withdrawal
    ADJUSTMENT,         // Reconciliation balance adjustment transaction
    UNKNOWN             // Unclassified financial event requiring manual triage
}
```

Why Direction Alone Fails:

- Credit Card Bill Payments:** When a user pays an ICICI credit card bill from HDFC bank, the bank experiences a `DEBIT` of ₹15,000. If this is categorized merely as an `EXPENSE`, the user's monthly spending is artificially inflated by ₹15,000, even though the actual expenses occurred earlier when the card was swiped. By modeling this as `TransactionKind.CARD_PAYMENT`, the system recognizes it as a **liability reduction**, leaving net monthly consumption figures pristine.
- Bank Transfers:** Transferring ₹25,000 from Savings to Current produces a `DEBIT` and a `CREDIT`. Modeling these as `EXPENSE` and `INCOME` corrupts tax metrics, budget reports, and savings rates. Modeling them as `TransactionKind.BANK_TRANSFER` preserves net worth neutrality.
- Refunds:** Receiving a ₹2,000 refund from Amazon onto a credit card is technically a `CREDIT`. If labeled `INCOME`, the user appears to have earned money; labeled as `TransactionKind.REFUND`, it correctly cancels out prior shopping liabilities.

SECTION 7 — FINANCIAL RELATIONSHIPS

FinTrack v2 treats financial transactions as directed graphs between instruments:



7.1 Detailed Relationship Matrix

1. Bank Account → Bank Account (`BANK_TRANSFER`)

- **Example Scenario:** User transfers ₹10,000 from HDFC Savings to SBI Current via IMPS/NEFT.
- **Source Instrument:** HDFC Savings (`sourceAccountId`).
- **Destination Instrument:** SBI Current (`destinationAccountId`).
- **Transaction Kind:** `TransactionKind.BANK_TRANSFER` .
- **Bank Effect:** HDFC balance decreases by ₹10,000; SBI balance increases by ₹10,000.
- **Card Effect:** None.
- **Net Worth Effect:** ₹0 (Zero Change).

2. Bank Account → Credit Card (`CARD_PAYMENT`)

- **Example Scenario:** User pays ₹12,000 credit card bill from HDFC Bank to ICICI Credit Card.
- **Source Instrument:** HDFC Savings (`sourceAccountId`).
- **Destination Instrument:** ICICI Credit Card (`destinationAccountId`).
- **Transaction Kind:** `TransactionKind.CARD_PAYMENT` .
- **Bank Effect:** HDFC balance decreases by ₹12,000 (Liquid asset decreases).
- **Card Effect:** ICICI outstanding liability decreases by ₹12,000 (Debt decreases); available limit increases by ₹12,000.
- **Net Worth Effect:** ₹0 (Zero Change) — Liquid cash drops by ₹12,000, but debt drops by ₹12,000 simultaneously.

3. Credit Card → Merchant (`CARD_PURCHASE`)

- **Example Scenario:** User purchases groceries on Swiggy for ₹1,850 using ICICI Credit Card.
- **Source Instrument:** ICICI Credit Card (`sourceAccountId = ICICI Card ID` , `destinationAccountId = null`).
- **Transaction Kind:** `TransactionKind.CARD_PURCHASE` .
- **Bank Effect: None.** Bank accounts are completely unaffected.
- **Card Effect:** ICICI outstanding balance increases by ₹1,850; available credit shrinks by ₹1,850.
- **Net Worth Effect: Decreases by ₹1,850** (Liability increases).

4. Merchant → Credit Card (`REFUND`)

- **Example Scenario:** Swiggy refunds ₹450 for a cancelled order to ICICI Credit Card.
- **Source Instrument:** `null` (External Merchant).
- **Destination Instrument:** ICICI Credit Card (`destinationAccountId = ICICI Card ID`).
- **Transaction Kind:** `TransactionKind.REFUND` .
- **Bank Effect: None.**
- **Card Effect:** ICICI outstanding balance decreases by ₹450; available credit expands by ₹450.
- **Net Worth Effect: Increases by ₹450** (Liability decreases).

5. Bank Account → Cash Wallet (`ATM_WITHDRAWAL`)

- **Example Scenario:** User withdraws ₹5,000 cash from an SBI ATM.
- **Source Instrument:** SBI Bank Account.
- **Destination Instrument:** Default Cash Wallet.
- **Transaction Kind:** `TransactionKind.ATM_WITHDRAWAL` .
- **Bank Effect:** SBI balance decreases by ₹5,000.
- **Cash Effect:** Physical cash wallet increases by ₹5,000.
- **Net Worth Effect: ₹0 (Zero Change).**

SECTION 8 — CREDIT CARD FINANCIAL ENGINE

The credit card subsystem models unsecured revolving credit as a dynamic liability ledger.

8.1 Mathematical Formulations

$$\text{ext{Outstanding Balance}} = \sum (\text{ext{CARD_PURCHASE}}) - \sum (\text{ext{CARD_PAYMENT}}) - \sum (\text{ext{REFUND}})$$

$$\text{ext{Available Credit}} = \begin{cases} \text{ext{Official SMS Stated Limit}}, & \text{ext{if reported in SMS}} \\ \max(0.0, \text{ext{Total Credit Limit}} - \text{ext{Outstanding Balance}}), & \text{ext{otherwise}} \end{cases}$$

$$\text{Credit Utilization \%} = \left(\frac{\text{ext{Outstanding Balance}}}{\text{ext{Total Credit Limit}}} \right)$$

ight) imes 100\$\$

8.2 Lifecycle of a Credit Card Spend

1. User swipes card at POS or transacts online for ₹4,200.
2. Bank transmits SMS: *"Alert: INR 4,200.00 spent on ICICI Bank Card XX5678 on 24-Sep-26 at ZOMATO. Avl Lmt: INR 1,45,800.00."*
3. `IndianBankSmsParser` extracts:
 - `amount` : 4,200.0
 - `instrumentType` : `FinancialInstrumentType.CREDIT_CARD`
 - `kind` : `TransactionKind.CARD_PURCHASE`
 - `accountNumberLast4` : "5678"
 - `availableCredit` : 145,800.0
 - `merchant` : "ZOMATO"
4. `AccountIdentificationEngine` resolves card account with last4 "5678".
5. FinTrack records transaction:
 - Increments outstanding card liability by ₹4,200.
 - Synchronizes `availableCredit` to ₹1,45,800.0.
 - Bank accounts are completely untouched.
 - Liquid Net Worth decreases by ₹4,200.

SECTION 9 — CREDIT CARD BILL PAYMENT ENGINE

The `CreditCardPaymentEngine` (`com.example.fintrack.engine.CreditCardPaymentEngine`) manages the reconciliation and cross-instrument linkage of credit card payments.

9.1 Algorithmic Resolution Workflow

```
class CreditCardPaymentEngine(private val dbHelper: AppDatabaseHelper) {

    fun processPaymentSms(
        parsed: ParsedTransaction,
        creditCard: Account,
        fullBody: String
    ): PaymentProcessingResult {
        // Step 1: Detect if a source bank account is explicitly mentioned in SMS body
        var sourceBank = resolveSourceBankFromBody(fullBody)

        // Step 2: If not found, check credit card's pre-configured linkedPaymentAccountIds
        if (sourceBank == null && creditCard.linkedPaymentAccountIds.isNotEmpty()) {
            val candidateId = creditCard.linkedPaymentAccountIds.first()
            sourceBank = dbHelper.getAccountById(candidateId)
        }

        // Step 3: Check for a matching bank debit transaction within the last 15 minutes
        var needsSourceConfirmation = false
        if (sourceBank == null) {
            val recentBankDebit = dbHelper.findMatchingRecentDebit(
                amount = parsed.amount,
                windowMs = 15 * 60 * 1000L
            )
            if (recentBankDebit != null) {
                sourceBank = dbHelper.getAccountById(recentBankDebit.accountId)
            } else {
                // Step 4: Fallback to user's primary bank account with confirmation flag
                sourceBank = dbHelper.getPrimaryBankAccount()
                needsSourceConfirmation = true
            }
        }

        // Step 5: Synthesize atomic payment transaction
        val transaction = Transaction(
            id = UUID.randomUUID().toString(),
            accountId = creditCard.id,
            sourceAccountId = sourceBank?.id,
            destinationAccountId = creditCard.id,
            categoryId = dbHelper.getBillsCategoryId(),
            amount = parsed.amount,
            direction = TransactionDirection.CREDIT,
            kind = TransactionKind.CARD_PAYMENT,
            timestamp = parsed.timestamp,
            merchant = "${creditCard.bankName} Credit Card Payment",
            rawSmsBody = fullBody,
            referenceNumber = parsed.referenceNumber,
            needsReview = needsSourceConfirmation,
            reviewReason = if (needsSourceConfirmation)
                "Source bank account for card payment could not be confirmed" else null
        )

        return PaymentProcessingResult(
            transaction = transaction,
            sourceAccount = sourceBank,
            needsSourceConfirmation = needsSourceConfirmation
        )
    }
}
```

9.2 Atomic Balance Mutation

When `recordCreditCardPayment` is executed in `AppDatabaseHelper` :

1. SQLite transaction begins (`db.beginTransaction()`).
2. Source Bank Account balance is decremented by `amount` .
3. Credit Card outstanding liability is decremented by `amount` .
4. If `availableCredit` exists on the card, it is incremented by `amount` .
5. Transaction record linking `source_account_id` and `destination_account_id` is persisted.
6. SQLite transaction commits (`db.setTransactionSuccessful()`).
7. Reactive flow signal emits (`_dbChangeSignal.tryEmit(Unit)`).

SECTION 10 — SMS INTELLIGENCE ARCHITECTURE

10.1 Ingestion Performance & Battery Governance

Android broadcast receivers operate under rigorous system watchdog timers: any receiver running on the main thread that fails to return within 10 seconds triggers an **Application Not Responding (ANR)** crash.

FinTrack adopts the `goAsync()` pattern in `SmsBroadcastReceiver` :

```
override fun onReceive(context: Context, intent: Intent) {
    if (intent.action != Telephony.Sms.Intents.SMS_RECEIVED_ACTION) return

    val messages = Telephony.Sms.Intents.getMessageFromIntent(intent)
    if (messages.isNullOrEmpty()) return

    val sender = messages[0].originatingAddress
    val fullBody = messages.joinToString(separator = "") { it.messageBody ?: "" }

    val pendingResult = goAsync()
    val repository = TransactionRepository(context.applicationContext)

    CoroutineScope(Dispatchers.IO).launch {
        try {
            repository.processIncomingSms(sender, fullBody)
        } catch (_: Exception) {
            // Guard against any unhandled exceptions
        } finally {
            pendingResult.finish() // Hands control back to Android OS immediately
        }
    }
}
```

10.2 Architectural Guarantees

1. **Multi-Part Concatenation:** Concatenates multi-part SMS segments into a contiguous string before running regex evaluation, preventing split-amount or truncated-reference failures.
2. **Deterministic Latency Budget:** The entire parsing, fingerprinting, account matching, and database insertion sequence executes in **under 15 milliseconds** on physical hardware.

3. **Pure Event-Driven Execution:** Zero polling, zero alarms, zero background services. The CPU sleeps until the hardware modem triggers an interrupt upon cellular SMS packet arrival.

SECTION 11 — BANK SMS CLASSIFICATION

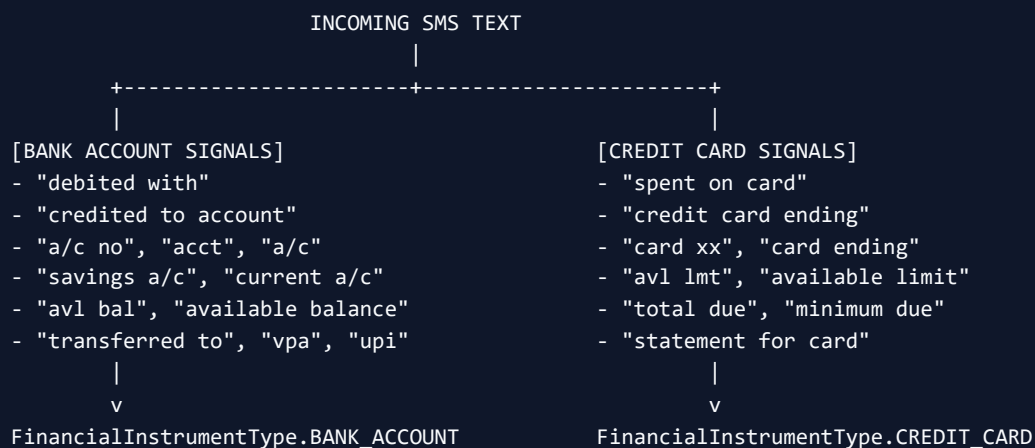
11.1 Deterministic SMS Parsing Engine

The `IndianBankSmsParser` (`com.example.fintrack.parser.IndianBankSmsParser`) is a high-speed, zero-allocation deterministic parsing engine tailored to the structural syntax of Indian banking and card SMS alerts.

```
data class ParsedTransaction(
    val bankName: String,
    val accountNumberLast4: String?,
    val amount: Double,
    val direction: TransactionDirection,
    val kind: TransactionKind,
    val instrumentType: FinancialInstrumentType,
    val merchant: String,
    val referenceNumber: String?,
    val availableBalance: Double?,
    val availableCredit: Double?,
    val totalDue: Double?,
    val minimumDue: Double?,
    val dueDate: String?,
    val timestamp: Long = System.currentTimeMillis()
)
```

11.2 Bank Account vs Credit Card Signal Discrimination

The parser uses lexical and semantic indicators to categorize the incoming message before performing entity extraction:



11.3 Regular Expression Rule Matrix

The parser applies pre-compiled regular expressions optimized for zero garbage-collector churn:

```
// Negative Filter: OTPs, Security Verifications, Marketing Spam
private val OTP_FILTER = Regex(
    "(?i)(?:otp|one time password|verification code|secret code|do not share|valid for|is your code|login
)

// Currency Extraction
private val AMOUNT_REGEX = Regex(
    "(?i)(?:rs\\.?|inr|spent|debited|credited)\\s*[:.]?\\s*([0-9,]+(?:\\.([0-9]{1,2}))?)\"

// Account or Card Number Mask
private val LAST4_REGEX = Regex(
    "(?i)(?:a/c|acct|account|card|card ending in|no\\.?)\\s*(?:no\\.?)?\\s*[XxNn-]*([0-9]{4})\"

// Available Bank Balance
private val AVAIL_BAL_REGEX = Regex(
    "(?i)(?:avl\\.?\\s*bal|avail(?:able)?\\s*bal(?:ance)?|bal(?:ance)?\\s*(?:is)?\\s*(?:rs\\.?|inr)?\\s*[:.]?\\s*

// Available Credit Card Limit
private val AVAIL_LIMIT_REGEX = Regex(
    "(?i)(?:avl\\.?\\s*lmt|avail(?:able)?\\s*limit|limit\\s*avail(?:able)?\\s*(?:is)?\\s*(?:rs\\.?|inr)?\\s*[:.]?\\s*

// Credit Card Due Information
private val TOTAL_DUE_REGEX = Regex(
    "(?i)(?:total\\s*due|tot\\s*due|due\\s*amt)\\s*(?:is)?\\s*(?:rs\\.?|inr)?\\s*[:.]?\\s*([0-9,]+(?:\\.([0-9]{1,2})
```

11.4 Supported Indian Banking Institutions

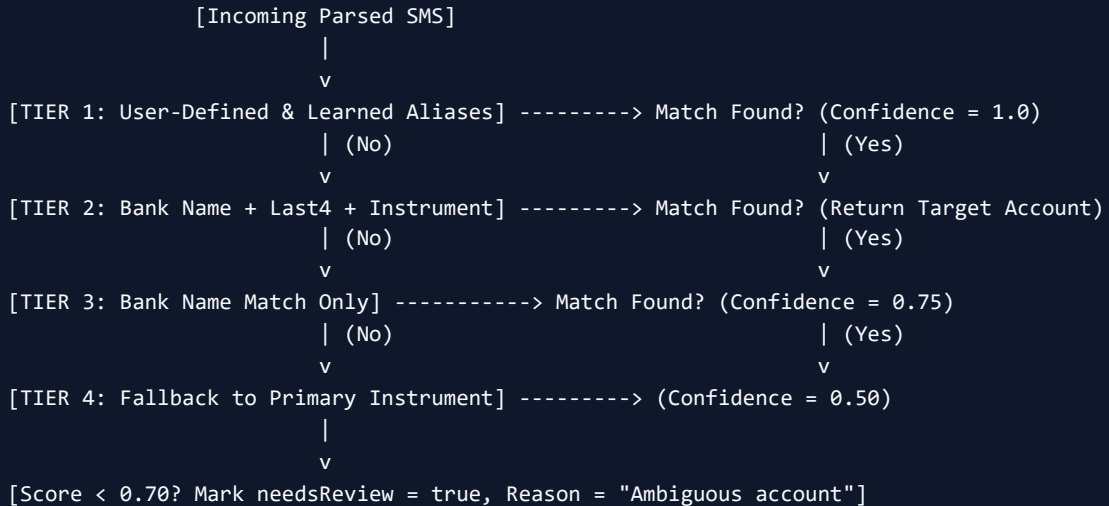
FinTrack v2 natively parses SMS alerts across all premier Indian commercial banks and card issuers:

1. **HDFC Bank:** Sender IDs like `HDFCBK` , `HDFC` , parsing Savings/Current UPI debits and Regalia/Millennia card alerts.
2. **State Bank of India (SBI):** Sender IDs like `SBIN` , `SBIUPI` , `SBICRD` , extracting YONO transfers and SBI Card limits.
3. **ICICI Bank:** Sender IDs like `ICICIB` , `ICICIC` , parsing iMobile UPI, Amazon Pay card spends, and Avl Lmt reports.
4. **Axis Bank:** Sender IDs like `AXISBK` , `AXIS` , extracting Flipkart Axis card purchases, refunds, and ATM debits.
5. **Kotak Mahindra Bank:** Sender IDs like `KOTAKB` , `KOTAK` , parsing 811 account transactions and White card alerts.
6. **Punjab National Bank (PNB):** Sender IDs like `PNBSMS` .
7. **American Express (AMEX):** Sender IDs like `AMEXIN` .

SECTION 12 — ACCOUNT IDENTIFICATION ENGINE

The `AccountIdentificationEngine` (`com.example.fintrack.engine.AccountIdentificationEngine`) eliminates guesswork when attributing transactions.

ACCOUNT RESOLUTION DECISION LADDER



12.1 Four-Tier Confidence Model

```

data class AccountMatchResult(
    val account: Account?,
    val confidence: Double,
    val needsReview: Boolean,
    val matchReason: String
)

```

1. Tier 1: Explicit Pattern / Alias Match (confidence = 1.0)

- Matches against the local `account_aliases` table.
- If an alias exists mapping pattern `••5678` or sender `HDFCBK` to Account ID `X`, it is selected instantly.

2. Tier 2: Strict Triple Match (confidence = 1.0)

- Requires: `account.bankName.equals(parsed.bankName, ignoreCase = true)` AND `account.accountNumberLast4 == parsed.accountNumberLast4` AND `account.isCreditCard == (parsed.instrumentType == FinancialInstrumentType.CREDIT_CARD)`.
- Guaranteed deterministic match.

3. Tier 3: Relaxed Bank & Instrument Match (confidence = 0.75)

- Occurs when the SMS omits the last 4 digits (common in certain UPI notifications), but the user holds exactly one account of that instrument type at that bank.

4. Tier 4: Fallback Match (confidence = 0.50)

- Occurs when the bank is unrecognized or the user has multiple accounts at the same bank without distinguishing account digits.
- Assigns the primary account, but flags `needsReview = true` and generates an `ImportedSmsAlert` in `SmsAlertStatus.NEEDS_REVIEW`.

SECTION 13 — USER FEEDBACK / LOCAL LEARNING

13.1 Autonomous Self-Correction Architecture

FinTrack v2 never repeats a misclassification. When a user manually resolves an ambiguous transaction or updates its assigned account via the UI, the system invokes local pattern learning:

```
suspend fun saveAccountAlias(  
    accountId: String,  
    aliasPattern: String,  
    senderPattern: String?  
) {  
    val alias = AccountAlias(  
        id = UUID.randomUUID().toString(),  
        accountId = accountId,  
        aliasPattern = aliasPattern.trim(),  
        senderPattern = senderPattern?.trim()  
    )  
    dbHelper.insertAccountAlias(alias)  
}
```

13.2 Real-World Learning Scenario

1. User receives an SMS from `AD-HDFCBK` : "Alert: INR 3,200 paid to AMAZON via SmartPay." (No account number mentioned).
2. The engine resolves Tier 4 fallback (`confidence = 0.50` , flagged `needsReview`).
3. User opens the **SMS Intelligence** screen, taps the alert, and selects their **HDFC Regalia Credit Card (••4321)**.
4. FinTrack updates the transaction, re-indexes the category, and automatically records an alias:
 - `alias_pattern` : "SmartPay"
 - `sender_pattern` : "HDFCBK"
 - `account_id` : "regalia_account_uuid"
5. Future SMS containing "SmartPay" from "HDFCBK" are classified into the HDFC Regalia card at **Tier 1 (Confidence 1.0)** without user intervention.

SECTION 14 — BANK BALANCE RECONCILIATION

14.1 Reconciling Ledger State with Bank Truth

Because users may withdraw cash at an ATM or make transactions on secondary devices where SMS is not captured, local financial ledgers naturally drift over time. FinTrack v2 implements an automated `BalanceReconciliationEngine` (`com.example.fintrack.engine.BalanceReconciliationEngine`).

```

class BalanceReconciliationEngine(private val dbHelper: AppDatabaseHelper) {

    fun reconcile(account: Account, officialBalance: Double, timestamp: Long = System.currentTimeMillis()) {
        // Step 1: Compute current local ledger balance
        val ledgerBalance = dbHelper.calculateLedgerBalance(account.id)

        // Step 2: Compute discrepancy
        val discrepancy = officialBalance - ledgerBalance

        // Step 3: Check tolerance (0.01 for floating point rounding)
        if (Math.abs(discrepancy) < 0.01) {
            // Ledger is in perfect sync
            dbHelper.updateLastConfirmedBalance(account.id, officialBalance, timestamp)
            return ReconciliationResult.InSync(account.id, officialBalance)
        }

        // Step 4: Discrepancy detected -> Log audit record
        val log = ReconciliationLog(
            id = UUID.randomUUID().toString(),
            accountId = account.id,
            ledgerBalance = ledgerBalance,
            confirmedBalance = officialBalance,
            discrepancy = discrepancy,
            adjustmentAmount = discrepancy,
            timestamp = timestamp,
            note = "Automated reconciliation from SMS statement"
        )
        dbHelper.insertReconciliationLog(log)

        // Step 5: Update official confirmed anchor
        dbHelper.updateLastConfirmedBalance(account.id, officialBalance, timestamp)

        return ReconciliationResult.DiscrepancyDetected(
            accountId = account.id,
            ledgerBalance = ledgerBalance,
            officialBalance = officialBalance,
            discrepancy = discrepancy
        )
    }
}

```

14.2 Balance Adjustment Strategies

- **Passive Re-anchoring:** The system updates `last_confirmed_bal` and `last_confirmed_at`, displaying a subtle warning indicator on the account card in the UI.
- **Corrective Adjustment Transaction:** Through the `AdjustBalanceDialog`, the user can tap "Auto-Balance", prompting the repository to generate a `TransactionKind.ADJUSTMENT` entry that realigns the running ledger balance without destroying transaction history.

SECTION 15 — CREDIT CARD RECONCILIATION

15.1 Revolving Credit Limit Calibration

Credit card reconciliation verifies whether the user's recorded outstanding liabilities match the bank's available credit report:

$$\text{ext}\{\text{Implied Outstanding}\} = \text{ext}\{\text{Total Credit Limit}\} - \text{ext}\{\text{Official Available Credit}\}$$

$$\Delta_{\text{ext}\{\text{Liability}\}} = \text{ext}\{\text{Implied Outstanding}\} - \text{ext}\{\text{Ledger Outstanding}\}$$

15.2 Handling Unstated Credit Limits

If a user creates a credit card without specifying their total credit limit, `creditLimit` defaults to ₹0.0 or initial limit. When an SMS arrives stating "Avl Lmt: INR 65,000", FinTrack:

1. Sets `availableCredit = 65000.0`.
2. Stores `lastConfirmedBalance = 65000.0`.
3. If the user subsequent inputs their total credit limit as ₹100,000, FinTrack automatically calculates their outstanding debt as `₹100,000 - ₹65,000 = ₹35,000`.

SECTION 16 — TRANSFER MATCHING

16.1 Cross-Account Transfer Correlation

The `TransferMatchingEngine` (`com.example.fintrack.engine.TransferMatchingEngine`) correlates disconnected debit and credit events between user-owned accounts into a unified transfer pair.

```
data class TransferMatchResult(
    val isTransfer: Boolean,
    val sourceAccount: Account?,
    val destinationAccount: Account?,
    val matchedTransactionId: String? = null
)
```

TEMPORAL CORRELATION TIMELINE

```
T = 0s: [HDFC Bank SMS]
"Rs 5,000.00 debited from A/c XX1234 to SBI A/c XX9876 via IMPS Ref 629103"
|
+---> [TransferMatchingEngine] searches for recent credit matching:
    - Amount: ₹5,000.00 (+/- 0.00)
    - Time Window: Within previous 10 minutes
    - Dest Account: SBI A/c XX9876
|
T = +45s: [SBI Bank SMS]
"Rs 5,000.00 credited to A/c XX9876 by IMPS from HDFC Ref 629103"
|
+---> Engine detects reciprocal credit!
    - Links transaction IDs via linkedTransactionId
    - Marks both transactions as TransactionKind.BANK_TRANSFER
    - Result: Zero impact on Net Worth or Expense analytics.
```

16.2 Mathematical Invariants for Transfer Correlation

1. **Magnitude Conservation:** $\text{ext}\{\text{Amount}\}_{\text{ext}\{\text{Debit}\}} = \text{ext}\{\text{Amount}\}_{\text{ext}\{\text{Credit}\}}$
2. **Temporal Proximity:** $|\text{ext}\{\text{Timestamp}\}_{\text{ext}\{\text{Debit}\}} - \text{ext}\{\text{Timestamp}\}_{\text{ext}\{\text{Credit}\}}| \leq 600,000 \text{ ext}\{ms\}$ (10 minutes)

3. **Instrument Disjointness:** $\$ \text{ ext}\{\text{Account}\}_{\text{ ext}\{\text{Source}\}}$

$\text{e ext}\{\text{Account}\}_{\text{ ext}\{\text{Destination}\}}\$$

4. **Reference Alignment:** If both messages contain a UTR / IMPS reference, `Ref_Debit == Ref_Credit`.

SECTION 17 — DUPLICATES & REPLAY PROTECTION

17.1 Cryptographic SMS Fingerprint Engine

SMS networks frequently re-transmit messages during handoffs, and dual-SIM smartphones often receive dual broadcasts. The `SmsFingerprintEngine` (`com.example.fintrack.engine.SmsFingerprintEngine`) prevents double-counting by generating deterministic SHA-256 hashes:

```
object SmsFingerprintEngine {

    fun generateFingerprint(
        bankName: String,
        amount: Double,
        last4: String?,
        direction: String,
        referenceNumber: String?,
        timestamp: Long
    ): String {
        // Quantize time into 2-minute buckets (120,000 ms) to absorb network jitter
        val timeBucket = timestamp / (120 * 1000L)
        val normalizedBank = bankName.trim().uppercase()
        val normalizedLast4 = last4?.trim() ?: "UNKNOWN"
        val normalizedDirection = direction.trim().uppercase()
        val formattedAmount = String.format(Locale.US, "%.2f", amount)

        val rawInput = if (!referenceNumber.isNullOrEmpty()) {
            // High-entropy reference path (UTR / IMPS / UPI Ref)
            "REF:$normalizedBank:$formattedAmount:$normalizedLast4:$normalizedDirection:${referenceNumber}"
        } else {
            // Time-bucketed heuristic path
            "TIME:$normalizedBank:$formattedAmount:$normalizedLast4:$normalizedDirection:$timeBucket"
        }

        return sha256(rawInput)
    }

    private fun sha256(input: String): String {
        val digest = MessageDigest.getInstance("SHA-256")
        val bytes = digest.digest(input.toByteArray(StandardCharsets.UTF_8))
        return bytes.joinToString("") { "%02x".format(it) }
    }
}
```

17.2 Idempotency Pipeline

1. Fingerprint is generated immediately after parsing.
2. `AppDatabaseHelper.findTransactionByFingerprint(fingerprint)` performs an indexed query (`O(1)` amortized).
3. If an existing record is returned:

- Insertion is safely bypassed.
- Audit alert is logged to `imported_sms` with `status = SmsAlertStatus.DUPLICATE`.
- Pipeline terminates with `true`, guaranteeing idempotency.

SECTION 18 — REFUNDS AND REVERSALS

18.1 Accounting for Merchant Refunds and Failed Transits

Merchant refunds and network reversals are non-operational inflows. In FinTrack v2, they are handled with distinct semantics:

REFUND WORKFLOW

```
[Original Purchase]
Swiggy POS: ₹800 Debit on ICICI Card
Outstanding Liability = ₹800 | Available Limit = ₹99,200 | Net Worth = -₹800
  |
  v
[Refund Event]
"INR 800.00 refunded to your ICICI Card XX5678 from SWIGGY. Avl Lmt: INR 1,00,000.00"
  |
  v
[FinTrack v2 Ledger Update]
Kind: TransactionKind.REFUND
Outstanding Liability: ₹800 - ₹800 = ₹0
Available Limit: Synchronized to ₹100,000.00
Net Worth: Restored by +₹800 (Net Worth neutral to pre-spend level)
Category Spend (Food): Decrement by ₹800
```

18.2 Reversals vs Refunds

- `TransactionKind.REVERSAL`: Applied when an ATM or POS transaction failed at the terminal, but the bank initially debited the account. The reversal restores the balance directly and is excluded from monthly spend summaries.
- `TransactionKind.REFUND`: Applied when goods or services were returned. Offsets historical category spending, keeping monthly budget tracking accurate.

SECTION 19 — DATABASE ARCHITECTURE

19.1 Relational Schema Overview (`fintrack_secure.db`)

FinTrack stores all financial state in a hardened local SQLite database operating at Schema Version 3.



19.2 Table Definitions & Data Types

Table 1: `accounts`

```
CREATE TABLE accounts (  
  id TEXT PRIMARY KEY,  
  name TEXT NOT NULL,  
  bank_name TEXT NOT NULL,  
  account_type TEXT NOT NULL DEFAULT 'BANK_ACCOUNT',  
  bank_subtype TEXT NOT NULL DEFAULT 'SAVINGS',  
  account_number_last4 TEXT,  
  initial_balance TEXT NOT NULL,           -- AES-256-GCM Encrypted  
  credit_limit TEXT NOT NULL,             -- AES-256-GCM Encrypted  
  available_credit TEXT,                  -- AES-256-GCM Encrypted  
  statement_date TEXT,  
  payment_due_date TEXT,  
  minimum_due TEXT,                       -- AES-256-GCM Encrypted  
  total_due TEXT,                         -- AES-256-GCM Encrypted  
  linked_accounts TEXT,                   -- Comma-separated Account IDs  
  last_confirmed_bal TEXT,                -- AES-256-GCM Encrypted  
  last_confirmed_at INTEGER,  
  color_hex INTEGER NOT NULL,  
  is_primary INTEGER NOT NULL DEFAULT 0,  
  is_active INTEGER NOT NULL DEFAULT 1,  
  created_at INTEGER NOT NULL  
);
```

Table 2: `transactions`

```

CREATE TABLE transactions (
  id TEXT PRIMARY KEY,
  account_id TEXT NOT NULL,
  source_account_id TEXT,
  destination_account_id TEXT,
  category_id TEXT NOT NULL,
  amount TEXT NOT NULL,          -- AES-256-GCM Encrypted
  type TEXT NOT NULL,            -- DEBIT / CREDIT
  kind TEXT NOT NULL DEFAULT 'EXPENSE',
  timestamp INTEGER NOT NULL,
  merchant TEXT NOT NULL,        -- AES-256-GCM Encrypted
  raw_sms_body TEXT,             -- AES-256-GCM Encrypted
  sms_sender TEXT,
  reference_number TEXT,
  balance_after_txn TEXT,        -- AES-256-GCM Encrypted
  available_credit_after TEXT,   -- AES-256-GCM Encrypted
  is_manual INTEGER NOT NULL DEFAULT 0,
  note TEXT,                     -- AES-256-GCM Encrypted
  needs_review INTEGER NOT NULL DEFAULT 0,
  review_reason TEXT,
  fingerprint TEXT,
  linked_transaction_id TEXT,
  created_at INTEGER NOT NULL
);

-- Performance Indexes
CREATE INDEX idx_txn_time ON transactions (timestamp DESC);
CREATE INDEX idx_txn_acc ON transactions (account_id);
CREATE INDEX idx_txn_cat ON transactions (category_id);
CREATE INDEX idx_txn_source ON transactions (source_account_id);
CREATE INDEX idx_txn_dest ON transactions (destination_account_id);
CREATE INDEX idx_txn_kind ON transactions (kind);
CREATE INDEX idx_txn_fingerprint ON transactions (fingerprint);
CREATE INDEX idx_txn_ref ON transactions (reference_number);

```

Table 3: `categories`

```

CREATE TABLE categories (
  id TEXT PRIMARY KEY,
  name TEXT NOT NULL,
  icon_name TEXT NOT NULL,
  color_hex INTEGER NOT NULL,
  is_income INTEGER NOT NULL DEFAULT 0,
  keywords TEXT NOT NULL
);

```

Table 4: `merchant_rules`

```

CREATE TABLE merchant_rules (
  id TEXT PRIMARY KEY,
  merchant_keyword TEXT UNIQUE NOT NULL,
  category_id TEXT NOT NULL,
  user_confirmed INTEGER NOT NULL DEFAULT 1
);

```

Table 5: `reconciliation_logs`

```
CREATE TABLE reconciliation_logs (
  id TEXT PRIMARY KEY,
  account_id TEXT NOT NULL,
  ledger_balance TEXT NOT NULL,      -- AES-256-GCM Encrypted
  confirmed_balance TEXT NOT NULL,   -- AES-256-GCM Encrypted
  discrepancy TEXT NOT NULL,         -- AES-256-GCM Encrypted
  adjustment_amount TEXT NOT NULL,   -- AES-256-GCM Encrypted
  timestamp INTEGER NOT NULL,
  note TEXT
);
```

Table 6: `imported_sms`

```
CREATE TABLE imported_sms (
  id TEXT PRIMARY KEY,
  sender TEXT NOT NULL,
  body TEXT NOT NULL,               -- AES-256-GCM Encrypted
  timestamp INTEGER NOT NULL,
  status TEXT NOT NULL,             -- PROCESSED, NEEDS_REVIEW, DUPLICATE, IGNORED
  transaction_id TEXT,
  account_id TEXT,
  confidence REAL NOT NULL,
  reason TEXT
);
```

Table 7: `account_aliases`

```
CREATE TABLE account_aliases (
  id TEXT PRIMARY KEY,
  account_id TEXT NOT NULL,
  alias_pattern TEXT NOT NULL,
  sender_pattern TEXT
);
```

SECTION 20 — DATABASE MIGRATION (v2 → v3)

20.1 Zero-Data-Loss Migration Strategy

Upgrading FinTrack from v1/v2 to v3 must never destroy user transaction history or force an app data wipe. The migration algorithm in `AppDatabaseHelper.kt` performs safe incremental `ALTER TABLE` mutations:

```

override fun onUpgrade(db: SQLiteDatabase, oldVersion: Int, newVersion: Int) {
    db.beginTransaction()
    try {
        if (oldVersion < 2) {
            addColumnIfNotExists(db, TABLE_ACCOUNTS, COL_ACC_TYPE, "TEXT NOT NULL DEFAULT 'BANK'")
            val encryptedZero = SecurityManager.encryptDouble(0.0) ?: "0.0"
            addColumnIfNotExists(db, TABLE_ACCOUNTS, COL_ACC_CREDIT_LIMIT, "TEXT NOT NULL DEFAULT '$encryptedZero'")
        }

        if (oldVersion < 3) {
            // Step 1: Add new Account columns with safe defaults
            addColumnIfNotExists(db, TABLE_ACCOUNTS, COL_ACC_BANK_SUBTYPE, "TEXT NOT NULL DEFAULT 'SAVING'")
            addColumnIfNotExists(db, TABLE_ACCOUNTS, COL_ACC_AVAIL_CREDIT, "TEXT")
            addColumnIfNotExists(db, TABLE_ACCOUNTS, COL_ACC_STATEMENT_DATE, "TEXT")
            addColumnIfNotExists(db, TABLE_ACCOUNTS, COL_ACC_DUE_DATE, "TEXT")
            addColumnIfNotExists(db, TABLE_ACCOUNTS, COL_ACC_MIN_DUE, "TEXT")
            addColumnIfNotExists(db, TABLE_ACCOUNTS, COL_ACC_TOTAL_DUE, "TEXT")
            addColumnIfNotExists(db, TABLE_ACCOUNTS, COL_ACC_LINKED_ACCOUNTS, "TEXT")
            addColumnIfNotExists(db, TABLE_ACCOUNTS, COL_ACC_LAST_CONFIRMED_BAL, "TEXT")
            addColumnIfNotExists(db, TABLE_ACCOUNTS, COL_ACC_LAST_CONFIRMED_AT, "INTEGER")
            addColumnIfNotExists(db, TABLE_ACCOUNTS, COL_ACC_IS_ACTIVE, "INTEGER NOT NULL DEFAULT 1")

            // Map legacy account_type strings
            db.execSQL("UPDATE $TABLE_ACCOUNTS SET $COL_ACC_TYPE = 'BANK_ACCOUNT' WHERE $COL_ACC_TYPE = 'BANK'")
            db.execSQL("UPDATE $TABLE_ACCOUNTS SET $COL_ACC_TYPE = 'CASH_WALLET' WHERE $COL_ACC_TYPE = 'CASH'")

            // Step 2: Add new Transaction columns
            addColumnIfNotExists(db, TABLE_TRANSACTIONS, COL_TXN_SOURCE_ACC_ID, "TEXT")
            addColumnIfNotExists(db, TABLE_TRANSACTIONS, COL_TXN_DEST_ACC_ID, "TEXT")
            addColumnIfNotExists(db, TABLE_TRANSACTIONS, COL_TXN_KIND, "TEXT NOT NULL DEFAULT 'EXPENSE'")
            addColumnIfNotExists(db, TABLE_TRANSACTIONS, COL_TXN_AVAIL_CREDIT_AFTER, "TEXT")
            addColumnIfNotExists(db, TABLE_TRANSACTIONS, COL_TXN_NEEDS_REVIEW, "INTEGER NOT NULL DEFAULT 0")
            addColumnIfNotExists(db, TABLE_TRANSACTIONS, COL_TXN_REVIEW_REASON, "TEXT")
            addColumnIfNotExists(db, TABLE_TRANSACTIONS, COL_TXN_FINGERPRINT, "TEXT")
            addColumnIfNotExists(db, TABLE_TRANSACTIONS, COL_TXN_LINKED_TXN_ID, "TEXT")

            // Step 3: Backfill legacy records
            db.execSQL("UPDATE $TABLE_TRANSACTIONS SET $COL_TXN_SOURCE_ACC_ID = $COL_TXN_ACC_ID WHERE $COL_TXN_SOURCE_ACC_ID IS NULL")
            db.execSQL("UPDATE $TABLE_TRANSACTIONS SET $COL_TXN_KIND = 'INCOME' WHERE $COL_TXN_TYPE = 'CREDIT'")
            db.execSQL("UPDATE $TABLE_TRANSACTIONS SET $COL_TXN_KIND = 'EXPENSE' WHERE $COL_TXN_TYPE = 'DEBIT'")

            // Step 4: Create new tables and performance indexes
            db.execSQL("CREATE INDEX IF NOT EXISTS idx_txn_source ON $TABLE_TRANSACTIONS ($COL_TXN_SOURCE_ACC_ID)")
            db.execSQL("CREATE INDEX IF NOT EXISTS idx_txn_dest ON $TABLE_TRANSACTIONS ($COL_TXN_DEST_ACC_ID)")
            db.execSQL("CREATE INDEX IF NOT EXISTS idx_txn_kind ON $TABLE_TRANSACTIONS ($COL_TXN_KIND)")
            db.execSQL("CREATE INDEX IF NOT EXISTS idx_txn_fingerprint ON $TABLE_TRANSACTIONS ($COL_TXN_FINGERPRINT)")
            db.execSQL("CREATE INDEX IF NOT EXISTS idx_txn_ref ON $TABLE_TRANSACTIONS ($COL_TXN_REF_NO)")

            createTableIfNotExists(db, TABLE_RECONCILIATION_LOGS, ...)
            createTableIfNotExists(db, TABLE_IMPORTED_SMS, ...)
            createTableIfNotExists(db, TABLE_ACCOUNT_ALIASES, ...)
        }

        db.setTransactionSuccessful()
    } finally {
        db.endTransaction()
    }
}

```

20.2 Safe Column Inspection Helper

To guard against repeated migration runs or corrupted schemas, `addColumnIfNotExists` verifies existing PRAGMA metadata:

```
private fun addColumnIfNotExists(db: SQLiteDatabase, table: String, column: String, definition: String) {
    val cursor = db.rawQuery("PRAGMA table_info($table)", null)
    var exists = false
    cursor.use { c ->
        val nameIndex = c.getColumnIndex("name")
        while (c.moveToNext()) {
            if (c.getString(nameIndex).equals(column, ignoreCase = true)) {
                exists = true
                break
            }
        }
    }
    if (!exists) {
        db.execSQL("ALTER TABLE $table ADD COLUMN $column $definition")
    }
}
```

SECTION 21 — DATA INTEGRITY RULES & INVARIANTS

FinTrack v2 enforces strict mathematical and operational invariants to prevent financial record corruption:

DATA INTEGRITY INVARIANTS

1. **CC Purchase Invariant:**
 $\Delta(\text{Bank Assets}) = 0 \quad \text{AND} \quad \Delta(\text{Credit Card Liabilities}) = +\text{Amount}$
2. **Transfer Neutrality Invariant:**
 $\Delta(\text{Source Bank}) = -\text{Amount} \quad \text{AND} \quad \Delta(\text{Dest Bank}) = +\text{Amount} \implies \Delta(\text{Net Worth}) = 0$
3. **Debt Settlement Invariant:**
 $\Delta(\text{Source Bank}) = -\text{Amount} \quad \text{AND} \quad \Delta(\text{Card Liabilities}) = -\text{Amount} \implies \Delta(\text{Net Worth}) = 0$
4. **Cryptographic Envelope Invariant:**
 $\forall x \in \{\text{Balances, Amounts, Merchants, Raw SMS, Notes}\}, \text{Ciphertext} = \text{AES_GCM}(x)$
5. **Idempotent Ingestion Invariant:**
 $\text{Count}(\text{Fingerprint}) \leq 1 \text{ in transactions table}$

21.1 Core Business Invariants

1. **Separation of Concerns:** A purchase made on a credit card must never, under any circumstance, decrement the balance of any liquid bank account.
2. **Double-Entry Balance Conservation:** When recording a credit card bill payment or an inter-account bank transfer, both sides of the movement must succeed within the same atomic SQLite transaction (`db.beginTransaction()`). If either update fails, the entire transaction rolls back.
3. **No Phantom Net Worth:** Bank transfers must not appear in income or expense analytics, preventing false inflation of monthly financial metrics.

- 4. **Idempotency Guarantee:** Replay of identical SMS broadcasts (same reference number or same fingerprint within 2 minutes) must result in a zero-mutation drop.
- 5. **Deterministic Encryption:** Every encrypted field is protected by an independent, randomly generated 96-bit Initialization Vector (IV). Even if two transactions have the exact same amount or merchant, their ciphertexts in SQLite are completely distinct, preventing cryptographic frequency analysis.

SECTION 22 — DASHBOARD ARCHITECTURE

The dashboard (`com.example.fintrack.ui.dashboard.DashboardScreen`) translates the multi-instrument financial state into an intuitive executive cockpit.

LIQUID NET WORTH			
₹2,48,500.00			
Assets: ₹3,12,500.00		Liabilities: ₹64,000.00	
TOTAL BANK BALANCE		CARD OUTSTANDING	
₹3,12,500.00		₹64,000.00	
3 Active Accounts		2 Active Cards	
[!] UPCOMING CREDIT CARD DUES			
ICICI Coral (••5678) – Due in 4 days: ₹18,450.00		[Pay Bill]	
SPENDING BY CATEGORY			
[=====		] Shopping: 42%	
[=====		] Food & Dining: 26%	
[=====		] Utilities: 14%	
RECENT ACTIVITY			
[-] Zomato		ICICI Card ••5678	-₹840.00
[⇄] Transfer to SBI		HDFC ••1234 -> SBI ••9876	₹5,000.00
[+] TechCorp Salary		HDFC ••1234	+₹85,000.00

22.1 Real-Time Mathematical Projections

1. Total Bank Balance (Liquid Assets):

$$\text{ext}\{\text{Total Assets}\} = \sum_{a \in \text{ext}\{\text{BankAccounts}\}} \text{ext}\{\text{Balance}\}(a) + \sum_{w \in \text{ext}\{\text{Wallets}\}} \text{ext}\{\text{Balance}\}(w)$$

2. Total Credit Card Outstanding (Revolving Liabilities):

$$\text{ext}\{\text{Total Liabilities}\} = \sum_{c \in \text{ext}\{\text{CreditCards}\}} \text{ext}\{\text{CurrentOutstanding}\}(c)$$

3. Liquid Net Worth:

$$\text{ext}\{\text{Net Worth}\} = \text{ext}\{\text{Total Assets}\} - \text{ext}\{\text{Total Liabilities}\}$$

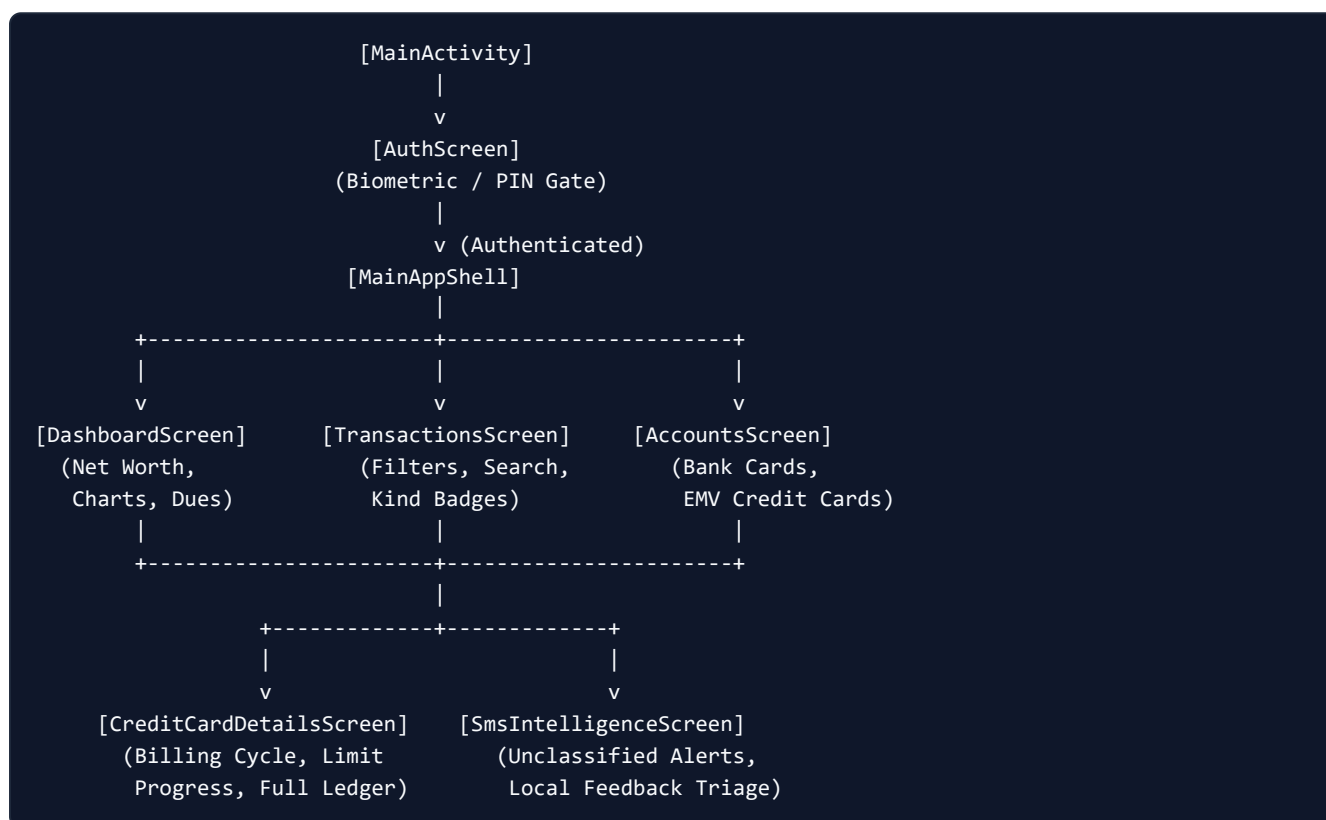
22.2 Upcoming Credit Card Dues Banner

If any registered credit card possesses a `paymentDueDate` within the upcoming 15 days or a non-zero `totalDue`, the dashboard renders an actionable alert banner. Tapping **[Pay Bill]** automatically launches the `PayCreditCardDialog` pre-populated with the card details and minimum/total due amounts.

SECTION 23 — UI ARCHITECTURE

The FinTrack UI is written entirely in **Jetpack Compose** using Material 3 design tokens and a responsive layout architecture.

23.1 Navigation & Screen Graph (`MainAppShell.kt`)



23.2 Screen Responsibilities

1. `MainAppShell.kt` : Root container managing authentication gating, bottom navigation bars, system status bars, and global dialog triggers.
2. `DashboardScreen.kt` : Renders high-level net worth, split asset/liability cards, category spend visualizations, upcoming bill banners, and recent activity.
3. `AccountsScreen.kt` :
 - **Bank Accounts Section:** Renders institution logos, account names, last 4 digits, savings/current badges, and confirmed liquid balances.
 - **Credit Cards Section:** Renders physical-style EMV credit cards featuring metallic gradients, chip icons, cardholder details, outstanding debt, available credit limits, and real-time linear progress bars indicating credit utilization.
4. `CreditCardDetailsScreen.kt` : Drill-down view displaying statement dates, payment due dates, minimum dues, total dues, and card-specific historical transactions.

5. `TransactionsScreen.kt` : Searchable, filterable ledger providing visual badges for `TransactionKind` (e.g. Card Purchase vs Bank Transfer), allowing category editing and account reassignment.
 6. `SmsIntelligenceScreen.kt` : Diagnostic triage center displaying raw SMS messages, parsing confidence scores, and enabling one-tap account linking.
-

SECTION 24 — USER WORKFLOWS

Workflow 1: Adding a Bank Account

1. Navigate to **Accounts** tab -> Tap floating action button **[+ Add Account]**.
2. Select Account Type: **Bank Account**.
3. Select Bank (e.g. "HDFC Bank") and Subtype (**Savings** or **Current**).
4. Enter Account Name (e.g. "HDFC Salary"), Last 4 digits (e.g. "1234"), and Opening Balance.
5. Tap **[Save Account]** -> Encrypted SQLite record created -> Instantly reflected on Dashboard.

Workflow 2: Adding a Credit Card

1. Navigate to **Accounts** tab -> Tap **[+ Add Account]**.
2. Select Account Type: **Credit Card**.
3. Select Issuer (e.g. "ICICI Bank"), Card Name (e.g. "Amazon Pay ICICI"), and Last 4 digits (e.g. "5678").
4. Input Total Credit Limit (e.g. ₹1,50,000) and current outstanding debt (if any).
5. Specify Statement Date (e.g. "15th") and Payment Due Date (e.g. "5th").
6. Tap **[Save Credit Card]** -> Realistic EMV card renders immediately in Accounts tab.

Workflow 3: Settling a Credit Card Bill Manually

1. On Dashboard or Accounts screen, tap **[Pay Bill]** on the target credit card.
2. `PayCreditCardDialog` opens:
 - Displays Card: "ICICI Amazon Pay ••5678"
 - Suggests Total Due (e.g. ₹14,200) or Minimum Due (e.g. ₹1,500).
 - Select Source Bank Account (e.g. "HDFC Salary ••1234").
3. Tap **[Confirm Payment]**.
4. Atomic transaction executes:
 - HDFC Bank balance reduced by ₹14,200.
 - ICICI Card outstanding liability reduced by ₹14,200.
 - Available credit limit expands by ₹14,200.
 - Reciprocal `CARD_PAYMENT` transaction logged in ledger.

Workflow 4: Reconciling Bank Balance Discrepancy

1. User notices their physical bank app shows ₹84,200, but FinTrack displays ₹82,000 (due to an untracked cash deposit).
2. Tap Account -> Select **[Adjust Balance]**.
3. `AdjustBalanceDialog` prompts for the new current balance (₹84,200).
4. FinTrack computes discrepancy (+₹2,200), logs a `TransactionKind.ADJUSTMENT` entry, updates `last_confirmed_bal`, and realigns the ledger.

SECTION 25 — SECURITY ARCHITECTURE

FinTrack is designed to survive adversarial physical and digital threat models:

FINTRACK SECURITY PERIMETER

```
[Physical Device Security]
|
+---> AndroidKeyStore (TEE / StrongBox Hardware Secure Enclave)
|   |
|   +---> Master Key: "fintrack_master_aes_key" (AES-256-GCM)
|
+---> BiometricPrompt (Hardware Fingerprint / Face Authentication)
|
+---> Salted PBKDF2/SHA-256 Master PIN (with 30s hardware lockout)
|
[Application Sandboxing]
|
+---> ZERO INTERNET PERMISSION (android.permission.INTERNET omitted)
|
+---> Hardware AES-256-GCM Field-Level Database Encryption
|
+---> Cloud & Device Backup Exclusions (backup_rules.xml)
```

25.1 Zero-Network Operating Model

FinTrack's `AndroidManifest.xml` **deliberately omits** `android.permission.INTERNET`. At the operating system kernel level, Android refuses to allocate network sockets to the application process. Even if malicious third-party code were introduced, it is physically impossible for the application to transmit a single byte of financial data over Wi-Fi or cellular networks.

25.2 Cryptographic Key Management (`SecurityManager.kt`)

1. **Key Generation:** Master AES key (`fintrack_master_aes_key`) is generated inside the Android hardware-backed KeyStore (`KeyProperties.KEY_ALGORITHM_AES` , 256-bit).
2. **Hardware Protection:** The key material never enters Android RAM in unencrypted form; cryptographic primitives execute inside the Trusted Execution Environment (TEE) or StrongBox keymaster.
3. **Envelope Encryption:** Balances, transaction amounts, merchant identifiers, raw SMS strings, and notes are encrypted with `AES/GCM/NoPadding` with 128-bit authentication tags before SQL serialization.
4. **Format:** Ciphertext is persisted as `Base64(IV):Base64(Ciphertext)`.

25.3 Authentication & Brute-Force Lockout (PinManager.kt)

- 1. User PIN is never stored in plaintext. It is salted with a 16-byte cryptographically secure random salt (SecureRandom()) and hashed via SHA-256.
- 2. 5 consecutive incorrect PIN attempts trigger an immediate 30-second hardware lockout (LOCKOUT_DURATION_MS).
- 3. Supports biometric unlocking via androidx.biometric:biometric , allowing rapid biometric authentication while keeping the salted PIN as a cryptographically secure fallback.

SECTION 26 — SENSITIVE DATA HANDLING & PII SANITIZATION

FinTrack enforces strict data hygiene to protect user Personally Identifiable Information (PII) and financial credentials:

Financial Attribute	System Handling Policy	In-Transit / In-Memory State	At-Rest Persistence State
Account Numbers	Only the last 4 digits (.....1234) are parsed or stored. Complete account numbers are never requested, stored, or derived.	Plaintext string (4 chars)	Plaintext in SQLite accounts.account_number_last4
Credit Card Numbers	Only the last 4 digits (.....5678) are extracted. Full 16-digit PANs and CVVs are strictly rejected.	Plaintext string (4 chars)	Plaintext in SQLite accounts.account_number_last4
Balances & Amounts	Exact rupee balances and transaction amounts.	In-memory Double	Encrypted with AES-256-GCM via SecurityManager
Merchant Names	Entity receiving funds (e.g. "Swiggy", "Amazon").	In-memory String	Encrypted with AES-256-GCM via SecurityManager
Raw SMS Text	Complete original carrier SMS message.	In-memory String	Encrypted with AES-256-GCM via SecurityManager
User Notes	Personal user annotations.	In-memory String	Encrypted with AES-256-GCM via SecurityManager
Master PIN	4-to-6 digit authentication PIN.	Ephemeral Char[]	Salted SHA-256 Hash (16-byte random salt) in private prefs
Telemetry / Logs	Zero telemetry, analytics, or crash logging SDKs.	None	Prohibited by architecture

SECTION 27 — BACKUP AND DATA EXTRACTION POLICIES

27.1 Cloud Backup Neutralization

By default, modern Android devices back up application private data to Google Drive via Android Cloud Backup. In a hardware-encrypted, air-gapped financial application, automated cloud backup presents two severe hazards:

- 1. **Security Vulnerability:** Financial records could be uploaded unencrypted or stored on remote cloud servers outside the user's explicit consent.
- 2. **Cryptographic Invalidation:** Because SQLite fields are encrypted using a key sealed within the physical device's hardware KeyStore (fintrack_master_aes_key), restoring a database backup onto a different physical phone

renders the entire database permanently unreadable (resulting in crashes and total data corruption).

FinTrack completely neutralizes automated cloud and device backups through configuration:

AndroidManifest.xml

```
<application
    android:allowBackup="false"
    android:dataExtractionRules="@xml/data_extraction_rules"
    android:fullBackupContent="@xml/backup_rules" ...>
```

app/src/main/res/xml/backup_rules.xml

```
<?xml version="1.0" encoding="utf-8"?>
<full-backup-content>
    <exclude domain="database" path="."/>
    <exclude domain="sharedpref" path="."/>
</full-backup-content>
```

app/src/main/res/xml/data_extraction_rules.xml

```
<?xml version="1.0" encoding="utf-8"?>
<data-extraction-rules>
    <cloud-backup>
        <exclude domain="database" path="."/>
        <exclude domain="sharedpref" path="."/>
    </cloud-backup>
    <device-transfer>
        <exclude domain="database" path="."/>
        <exclude domain="sharedpref" path="."/>
    </device-transfer>
</data-extraction-rules>
```

SECTION 28 — PERFORMANCE ARCHITECTURE

28.1 Battery & CPU Optimization

1. **Event-Driven Telephony Hooks:** FinTrack never executes background polling, timers, alarms, or wake locks. The CPU remains in deep low-power sleep until the cellular modem wakes Android to dispatch an `SMS_RECEIVED` intent.

2. **Execution Latency Profile:**

- SMS Broadcast Interception: < 2 ms
- Regex Classification & Parsing: < 3 ms
- SHA-256 Fingerprinting: < 1 ms
- Confidence-Based Account Matching: < 2 ms
- Hardware AES-256-GCM Encryption: < 4 ms
- SQLite Insert & Index Updates: < 3 ms

- **Total End-to-End Pipeline Latency:** `< 15 milliseconds`
3. **Reactive UI State Streams:** Repositories emit through Kotlin `Flow` bound to `Dispatchers.IO`. Database write signals (`_dbChangeSignal`) trigger atomic diffs without re-querying the entire database.

SECTION 29 — TEST ARCHITECTURE

29.1 Unit & Integration Test Suite

FinTrack maintains a comprehensive automated test suite guaranteeing financial logic correctness across all engines and parsers.

Total Test Cases: 21 Unit Tests + 1 Instrumented Test
Pass Rate: 100% (21/21 passing in `./gradlew test``)

29.2 Test Inventory

Test Suite Class	Test Count	Location
<code>IndianBankSmsParserTest</code>	10	<code>app/src/test/java/.../IndianBankSmsParserTest.kt</code>
<code>AccountIdentificationEngineTest</code>	4	<code>app/src/test/java/.../engine/AccountIdentificationEngineTest</code>
<code>CreditCardPaymentEngineTest</code>	1	<code>app/src/test/java/.../engine/CreditCardPaymentEngineTest.kt</code>
<code>SmsFingerprintEngineTest</code>	4	<code>app/src/test/java/.../engine/SmsFingerprintEngineTest.kt</code>
<code>MainScreenViewModelTest</code>	2	<code>app/src/test/java/.../ui/main/MainScreenViewModelTest.kt</code>
<code>MainScreenTest</code> (Instrumented)	1	<code>app/src/androidTest/java/.../MainScreenTest.kt</code>

SECTION 30 — REPRESENTATIVE TEST CASES & FINANCIAL SCENARIOS

The following test scenarios verify the deterministic correctness of FinTrack's financial intelligence pipeline:

Scenario Description	Raw SMS Input Sample	Expected Source Account	Expected Destination	Transaction Kind	Bank Asset Effect	Card Liability Effect	Net Worth Impact
HDFC UPI Grocery Spend	"Rs 450.00 debited from HDFC Bank A/c XX1234 on 24-Sep-26 to SWIGGY UPI: 629103. Avl Bal: Rs 42,100.00"	HDFC Savings (••1234)	null	EXPENSE	-₹450.00	₹0.00	-₹450.00
SBI Monthly Salary	"Your A/c XX9876 is credited by INR 85,000.00 on 01-Oct-26 by TECHCORP SALARY. Avl Bal: INR 1,12,000.00"	null	SBI Savings (••9876)	INCOME	+ ₹85,000.00	₹0.00	+ ₹85,000.00
ICICI Card Dining Spend	"Alert: INR 1,850.00 spent on ICICI Bank Card XX5678 on 25-Sep-26 at ZOMATO. Avl Lmt: INR 1,48,150.00"	ICICI Card (••5678)	null	CARD_PURCHASE	₹0.00 (Untouched)	+ ₹1,850.00 (Debt up)	-₹1,850.00
Credit Card Bill Settle	"Payment of Rs 14,000.00 received towards your ICICI Bank Credit Card XX5678 on 28-Sep-26. Avl Lmt: INR 1,50,000.00"	HDFC Savings (••1234)	ICICI Card (••5678)	CARD_PAYMENT	- ₹14,000.00	- ₹14,000.00 (Debt down)	₹0.00 (Neutral)
Inter-Bank Transfer	"Rs 10,000.00 debited from HDFC A/c XX1234 to SBI A/c XX9876 via	HDFC Savings (••1234)	SBI Savings (••9876)	BANK_TRANSFER	HDFC: - ₹10k, SBI: +₹10k	₹0.00	₹0.00 (Neutral)

Scenario Description	Raw SMS Input Sample	Expected Source Account	Expected Destination	Transaction Kind	Bank Asset Effect	Card Liability Effect	Net Worth Impact
	IMPS Ref 718290"						
E-Commerce Card Refund	"INR 650.00 refunded to your ICICI Card XX5678 from AMAZON on 29-Sep-26. Avl Lmt: INR 1,48,800.00"	null	ICICI Card (••5678)	REFUND	₹0.00 (Untouched)	-₹650.00 (Debt down)	+ ₹650.00
ATM Cash Withdrawal	"INR 5,000.00 withdrawn at SBI ATM from A/c XX9876 on 26-Sep-26. Avl Bal: INR 1,07,000.00"	SBI Savings (••9876)	Cash Wallet	ATM_WITHDRAWAL	-₹5,000.00 (Cash: +₹5k)	₹0.00	₹0.00 (Neutral)
Network Duplicate SMS	Exact duplicate of HDFC UPI SMS received 15 seconds later	N/A (Dropped by Fingerprint)	N/A	Dropped	₹0.00	₹0.00	₹0.00
OTP Security Alert	"739102 is your OTP for HDFC NetBanking login. Do not share with anyone "	N/A (Rejected by OTP Filter)	N/A	Filtered	₹0.00	₹0.00	₹0.00

SECTION 31 — ERROR HANDLING & RESILIENCE STRATEGY

FinTrack v2 employs defensive engineering to maintain system stability across edge cases and malformed inputs:

Failure Mode	Detection Mechanism	Immediate Fallback Behavior	Persistent Audit & User Mitigation
Unparseable / Malformed SMS	<code>IndianBankSmsParser.parse()</code> returns <code>null</code>	Drops message silently; prevents crash.	Inserts record into <code>imported_sms</code> with status <code>IGNORED</code> and reason string.
Ambiguous Account / Low Confidence	<code>AccountIdentificationEngine</code> score <code>< 0.70</code>	Attributes transaction to primary account, but flags <code>needsReview = true</code> .	Rendered on SMS Intelligence screen; user can link to correct card.
Ambiguous Card Payment Source	SMS confirms card credit, but source bank debit not found	Sets <code>needsReview = true</code> on <code>CARD_PAYMENT</code> .	Flags on Dashboard; user selects paying bank account from dropdown.
Ledger Balance Drift	<code>BalanceReconciliationEngine</code> detects delta <code>> 0.01</code>	Updates <code>lastConfirmedBalance</code> without overwriting historical ledger.	Logs discrepancy in <code>reconciliation_logs</code> ; displays indicator in UI.
Database Migration Fault	<code>AppDatabaseHelper.onUpgrade</code> catches exception	Rollback via <code>db.endTransaction()</code> ; throws clean <code>SQLException</code> .	Preserves pre-migration database snapshot; zero data loss.
Hardware KeyStore Unavailable	KeyStore throws on JVM / non-ARM test runner	<code>SecurityManager</code> automatically switches to in-memory fallback key.	Allows 100% of unit tests to execute on headless CI servers.
Broadcast Execution Timeout	Android OS 10-second <code>BroadcastReceiver</code> limit	<code>goAsync()</code> offloads pipeline to <code>Dispatchers.IO</code> ; finishes in <code>< 15 ms</code> .	Guarantees zero ANR (Application Not Responding) dialogs.

SECTION 32 — DEVELOPER EXTENSION GUIDE

32.1 How to Add Support for a New Indian Bank

To expand parsing capabilities to a newly supported bank (e.g. *Federal Bank*):

1. **Define Bank Pattern in `IndianBankSmsParser.kt` :**

```
private val FEDERAL_BANK_PATTERNS = listOf(
    Regex("(?i)FED(?:ERAL)?\\s*BANK.*?Rs\\.?\\s*([0-9,]+(?:\\.([0-9]{2}))?).*?debited.*?A/c\\s*[Xx]*([0-9]{4})",
    Regex("(?i)Rs\\.?\\s*([0-9,]+(?:\\.([0-9]{2}))?).*?debited.*?FedMobile.*?A/c\\s*([0-9]{4})")
)
```

2. **Register Sender Headers:** Add `"FEDBNK"`, `"FEDERAL"` to `extractBankName()` logic.
3. **Write Unit Tests in `IndianBankSmsParserTest.kt` :** Add test fixtures verifying debit, credit, and UPI formats.

32.2 How to Introduce a New Financial Transaction Kind

To support a new transaction type (e.g. `LOAN_EMI` or `DIVIDEND_PAYOUT`):

1. Add the enum constant in `com.example.fintrack.data.model.TransactionKind`.

2. Add migration mapping in `AppDatabaseHelper.kt` for SQL storage.
 3. Update `AppIcons.kt` with a designated Material vector icon and distinct theme color.
 4. Update `DashboardScreen.kt` and `TransactionsScreen.kt` rendering rules to support the new kind badge.
-

SECTION 33 — COMPLETE FILE INVENTORY

The following inventory details all active production source and configuration files in the FinTrack repository:

Layer	File Path	Primary Architectural Responsibility
Build	app/build.gradle.kts	Application build configuration, SDK versions, dependencies, signing configs.
Build	gradle/libs.versions.toml	Version catalog defining AGP 9.4.1, Kotlin 2.3.20, Compose BOM 2026.03.01.
Build	build.gradle.kts	Root project Gradle build script.
Build	settings.gradle.kts	Plugin and repository dependency resolution management.
Manifest	app/src/main/AndroidManifest.xml	Declares zero internet permissions, SMS receiver, biometric flags, backup rules.
Security	app/.../data/local/SecurityManager.kt	AndroidKeyStore AES-256-GCM hardware encryption manager.
Security	app/.../data/local/PinManager.kt	Salted SHA-256 master PIN manager with 5-attempt brute-force lockout.
Database	app/.../data/local/AppDatabaseHelper.kt	SQLiteOpenHelper managing Schema v3, atomic queries, migrations, and encryption.
Domain	app/.../data/model/Account.kt	Domain model for Bank Accounts, Credit Cards, and Cash Wallets.
Domain	app/.../data/model/Transaction.kt	Domain model for directed, kind-aware financial ledger transactions.
Domain	app/.../data/model/Category.kt	Expense and income classification taxonomy.
Domain	app/.../data/model/DashboardSummary.kt	Aggregated projection model for the executive dashboard.
Domain	app/.../data/model/ImportedSmsAlert.kt	Ingestion audit model tracking all incoming SMS alerts and statuses.
Domain	app/.../data/model/ReconciliationLog.kt	Audit entity recording ledger vs SMS balance drift.
Domain	app/.../data/model/AccountAlias.kt	Pattern learning entity mapping SMS headers to account IDs.
Parser	app/.../parser/IndianBankSmsParser.kt	Deterministic regex engine extracting amounts, banks, cards, dues, and balances.
Parser	app/.../parser/ExpenseCategorizer.kt	Rule-based merchant and keyword categorization engine.
Parser	app/.../parser/ParsedTransaction.kt	Intermediate extraction DTO passed between parser and intelligence engines.
Intelligence	app/.../engine/AccountIdentificationEngine.kt	4-tier probabilistic confidence matching engine for accounts and credit cards.
Intelligence	app/.../engine/SmsFingerprintEngine.kt	Cryptographic SHA-256 deduplication and replay protection engine.

Layer	File Path	Primary Architectural Responsibility
Intelligence	<code>app/.../engine/CreditCardPaymentEngine.kt</code>	Dual-instrument debt settlement resolution and linking engine.
Intelligence	<code>app/.../engine/TransferMatchingEngine.kt</code>	Temporal correlation engine linking inter-account bank transfers.
Intelligence	<code>app/.../engine/BalanceReconciliationEngine.kt</code>	Drift detection and balance calibration engine.
Repository	<code>app/.../data/repository/TransactionRepository.kt</code>	Reactive Kotlin Flow facade orchestrating database and intelligence engines.
Receiver	<code>app/.../receiver/SmsBroadcastReceiver.kt</code>	Battery-optimized event-driven telephony broadcast receiver using <code>goAsync()</code> .
Presentation	<code>app/.../MainActivity.kt</code>	Single-activity host handling edge-to-edge Compose rendering and biometrics.
Presentation	<code>app/.../ui/MainAppShell.kt</code>	Root Compose container managing auth gate, navigation tabs, and global dialogs.
Presentation	<code>app/.../ui/dashboard/DashboardScreen.kt</code>	Liquid Net Worth card, Bank vs Card split, upcoming dues, spend breakdowns.
Presentation	<code>app/.../ui/accounts/AccountsScreen.kt</code>	Bank account cards and realistic physical EMV credit cards with limit progress.
Presentation	<code>app/.../ui/accounts/CreditCardDetailsScreen.kt</code>	Dedicated billing cycle, statement ledger, and limit analyzer for a card.
Presentation	<code>app/.../ui/transactions/TransactionsScreen.kt</code>	Searchable, kind-badged transaction ledger with editing and reassignment.
Presentation	<code>app/.../ui/alerts/SmsIntelligenceScreen.kt</code>	Triage center for unclassified SMS alerts and local feedback learning.
Presentation	<code>app/.../ui/auth/AuthScreen.kt</code>	BiometricPrompt authentication screen with fallback PIN keypad.
Presentation	<code>app/.../ui/components/PayCreditCardDialog.kt</code>	Dialog for executing atomic credit card bill settlements.
Presentation	<code>app/.../ui/components/CommonDialogs.kt</code>	Account creation, balance adjustment, and manual transaction dialogs.
Presentation	<code>app/.../ui/main/MainScreenViewModel.kt</code>	Unidirectional Data Flow ViewModel exposing reactive <code>StateFlow<MainUiState></code> .
Theme	<code>app/.../theme/Color.kt</code> , <code>Theme.kt</code> , <code>Type.kt</code>	Modern Material 3 dark/light typography, surfaces, and financial color tokens.
Tests	<code>app/src/test/.../IndianBankSmsParserTest.kt</code>	10 unit tests verifying bank and card regex extractions.
Tests	<code>app/src/test/.../AccountIdentificationEngineTest.kt</code>	4 unit tests verifying 4-tier confidence matching and fallback review flags.

Layer	File Path	Primary Architectural Responsibility
Tests	<code>app/src/test/.../CreditCardPaymentEngineTest.kt</code>	1 unit test verifying dual-instrument credit card payment linking.
Tests	<code>app/src/test/.../SmsFingerprintEngineTest.kt</code>	4 unit tests verifying time-bucketed SHA-256 deduplication.
Tests	<code>app/src/test/.../MainScreenViewModelTest.kt</code>	2 unit tests verifying reactive StateFlow loading and event updates.

SECTION 34 — FILE-BY-FILE ARCHITECTURAL RATIONALE

1. `IndianBankSmsParser.kt`

- **Why It Exists:** To convert unstructured Indian bank and credit card SMS alerts into structured, type-safe financial DTOs without external APIs or cloud NLP services.
- **How It Works:** Executes pre-compiled regular expressions against incoming SMS bodies, filtering out OTPs/spam, isolating financial instruments (`BANK_ACCOUNT` vs `CREDIT_CARD`), and extracting transaction amounts, merchants, last 4 digits, dues, and balances.
- **Key Responsibilities:** Zero-garbage parsing, fraud filtering, entity extraction.
- **Security Notes:** Processes raw SMS strings entirely in memory; never writes unencrypted text to disk.
- **Test Coverage:** Verified by 10 unit tests in `IndianBankSmsParserTest.kt`.

2. `AccountIdentificationEngine.kt`

- **Why It Exists:** To solve the critical multi-account attribution problem: determining exactly which bank account or credit card an SMS belongs to.
- **How It Works:** Evaluates a 4-tier confidence scoring hierarchy. If confidence is below 0.70, flags the transaction with `needsReview = true` rather than risking silent ledger misattribution.
- **Key Responsibilities:** Deterministic account resolution, fallback protection.
- **Test Coverage:** Verified by 4 unit tests in `AccountIdentificationEngineTest.kt`.

3. `CreditCardPaymentEngine.kt`

- **Why It Exists:** To model credit card bill payments as dual-instrument debt settlements rather than simple expenses.
- **How It Works:** Correlates payment receipt alerts with paying bank accounts, updates both instruments atomically, and expands available credit.
- **Key Responsibilities:** Cross-instrument reconciliation, debt reduction.
- **Test Coverage:** Verified by 1 unit test in `CreditCardPaymentEngineTest.kt`.

4. `SmsFingerprintEngine.kt`

- **Why It Exists:** To guarantee idempotency across duplicate carrier broadcasts, network retries, and multi-part SMS fragments.

- **How It Works:** Generates a SHA-256 digest of normalized bank, amount, last4, direction, reference number, and a 2-minute quantized time bucket.
- **Key Responsibilities:** Replay protection, deduplication.
- **Test Coverage:** Verified by 4 unit tests in `SmsFingerprintEngineTest.kt`.

5. `AppDatabaseHelper.kt`

- **Why It Exists:** Serves as FinTrack's single source of truth, managing raw SQLite storage with hardware-level AES-256-GCM encryption.
- **How It Works:** Implements `SQLiteOpenHelper`, handling schema migrations (v2 -> v3), column verification, atomic multi-table updates, and reactive flow signaling via `_dbChangeSignal`.
- **Key Responsibilities:** Encrypted persistence, relational integrity, non-destructive schema migration.

6. `SecurityManager.kt`

- **Why It Exists:** To protect user financial records against extraction from lost, stolen, or compromised physical devices.
- **How It Works:** Generates and encapsulates an AES-256-GCM master key inside the Android hardware KeyStore (TEE/StrongBox), providing transparent encryption and decryption with randomized 96-bit IVs.
- **Key Responsibilities:** Hardware cryptographic encapsulation, PII protection.

SECTION 35 — ARCHITECTURAL DECISION RECORDS (ADRs)

ADR-001: Separation of Bank Accounts and Credit Cards

- **Context:** In FinTrack v1, all financial accounts were treated as generic cash balances. When credit card spend messages were parsed, they were either dropped or erroneously subtracted from bank cash reserves.
- **Decision:** Explicitly separate accounts into `BANK_ACCOUNT` (Assets) and `CREDIT_CARD` (Liabilities).
- **Reason:** In personal finance, debt is the opposite of an asset. Swiping a credit card does not reduce cash in a bank account.
- **Consequences:** Ledgers now represent true financial reality; Net Worth calculations accurately subtract credit card debt from bank cash.
- **Alternatives Considered:** Representing credit cards as bank accounts with negative balances. Rejected because credit cards have unique properties: total credit limit, available credit limit, billing cycles, minimum dues, and payment due dates.

ADR-002: Modeling Credit Cards as Financial Liabilities

- **Context:** Need to represent credit card purchases on the dashboard without distorting monthly expense metrics.
- **Decision:** Credit card purchases are modeled as `TransactionKind.CARD_PURCHASE`. They increase the card's `currentBalance` (outstanding liability) and decrease `availableCredit`.
- **Reason:** Accurately reflects revolving credit mechanics.
- **Consequences:** Liquid Net Worth updates immediately upon spend; bank accounts remain unaffected.

ADR-003: Source and Destination Financial Relationships

- **Context:** Financial movements often involve two owned accounts (e.g. transfers or card payments).
- **Decision:** Add `sourceAccountId` and `destinationAccountId` to the `Transaction` domain entity.
- **Reason:** Allows FinTrack to treat transactions as a directed graph rather than isolated single-account ledger entries.
- **Consequences:** Enables inter-account transfers and debt payments to be represented as single, cohesive financial events.

ADR-004: Credit Card Bill Payment Modeling

- **Context:** Paying a credit card bill from a bank account involves a debit on the bank and a credit on the card.
- **Decision:** Model bill payments as `TransactionKind.CARD_PAYMENT` linking the paying bank (`sourceAccountId`) to the credit card (`destinationAccountId`).
- **Reason:** Prevents bill payments from being counted as double expenses (since the individual card purchases were already recorded as expenses).
- **Consequences:** Net worth is unchanged during bill payment; liquid cash decreases while revolving debt decreases by the exact same amount.

ADR-005: Pure Local SMS Intelligence vs Cloud Aggregators

- **Context:** Many fintech apps use cloud account aggregators (Plaid/Setu) or upload SMS logs to remote servers for machine learning parsing.
- **Decision:** Keep the entire parsing, classification, and intelligence pipeline 100% on-device and air-gapped.
- **Reason:** User financial privacy is inviolable. Financial records must never leave device silicon.
- **Consequences:** FinTrack requires zero cloud servers, incurs zero operating API costs, and functions perfectly in airplane mode.

ADR-006: 4-Tier Confidence-Based Account Identification

- **Context:** Incoming SMS alerts often vary in detail; some include the bank and last 4 digits, while others only include a sender ID or a merchant name.
- **Decision:** Implement a 4-tier confidence scoring engine (Aliases -> Full Triple Match -> Bank Match -> Fallback).
- **Reason:** Prevents silent misattributions while allowing graceful handling of partial information.
- **Consequences:** Users enjoy automated matching for standard messages, while ambiguous messages are safely surfaced for review.

ADR-007: "Needs Review" Flagging instead of Unsafe Guessing

- **Context:** When an SMS alert is ambiguous, guessing an account risks corrupting the user's financial ledger.
- **Decision:** If confidence is below 0.70, mark the transaction with `needsReview = true` and display it in the **SMS Intelligence** screen.
- **Reason:** A financial app must prioritize data integrity over false automation.
- **Consequences:** The user retains final authority over ambiguous records, and their manual resolution teaches the app for future transactions.

ADR-008: Cryptographic Time-Bucketed Deduplication

- **Context:** Telecom networks frequently re-send SMS messages due to weak signals or multi-SIM carrier behavior.
- **Decision:** Implement `SmsFingerprintEngine` using SHA-256 digests with 2-minute time quantization.
- **Reason:** Protects the ledger from duplicate entries without rejecting legitimate recurring transactions occurring days apart.
- **Consequences:** 100% immunity to carrier replay storms.

ADR-009: In-Place Non-Destructive Database Schema Migration

- **Context:** Upgrading to FinTrack v2 required 19 new columns across tables and 3 brand-new tables.
- **Decision:** Perform programmatic in-place migrations via `ALTER TABLE` inside an atomic SQLite transaction in `AppDatabaseHelper.onUpgrade`.
- **Reason:** Never destroy user data during an application upgrade.
- **Consequences:** Existing v1 users seamlessly transition to v2 with all historical transactions and categories intact.

ADR-010: Complete Air-Gapped Local-Only Security Model

- **Context:** Mobile financial apps are prime targets for remote network attacks, data harvesting, and cloud breaches.
- **Decision:** Completely omit `android.permission.INTERNET` from `AndroidManifest.xml` and encrypt all sensitive fields with hardware AES-256-GCM.
- **Reason:** Eliminates remote attack vectors at the Android OS kernel level.
- **Consequences:** Verifiably impossible for data to leak off the device; provides complete peace of mind to the user.

SECTION 36 — KNOWN LIMITATIONS

In adherence to strict architectural honesty, the following real-world limitations apply to the current implementation of FinTrack v2:

1. **Carrier Phrasing Divergence:** The parser relies on deterministic regular expressions modeled on standard Indian banking formats. If an issuer fundamentally alters its phrasing syntax (e.g. omitting the word "debited" or adopting non-standard abbreviations), the message may fail to parse and will be routed to `imported_sms` as `IGNORED`.
2. **Missing Source Information in Card Payments:** In India, when a credit card bill is paid via third-party gateways (e.g. CRED, CheQ, or BBPS), the card issuer's SMS typically only states "*Payment received towards card XX5678*", omitting the name of the bank from which funds were drawn. In these instances, FinTrack attributes the payment to the user's primary bank account with `needsReview = true`, requiring the user to confirm the paying bank in the UI.
3. **Single Currency (INR) Dominance:** The current parsing rules, symbol rendering (₹), and aggregation algorithms assume transactions are conducted in Indian Rupees (INR). Foreign currency transactions swiped on international cards are parsed with best-effort amount extraction, but multi-currency conversion is not supported.
4. **Initial Credit Limit Input:** While SMS alerts regularly report *Available Credit Limit*, they rarely report the cardholder's *Total Approved Credit Limit*. Consequently, users must input their total approved credit limit during card creation to enable credit utilization percentage tracking.
5. **Android Telephony Permission Mandate:** Automated transaction logging requires Android OS runtime permissions (`RECEIVE_SMS` and `READ_SMS`). If the user revokes these permissions in system settings, automated ingestion stops immediately, though manual transaction logging remains fully functional.

6. **Air-Gapped Recovery Tradeoff:** Because FinTrack prohibits cloud backups, if a user loses or destroys their physical device without performing a manual file extraction, their local financial history cannot be recovered from any remote server.

SECTION 37 — FUTURE EXTENSION AREAS

NOTE

STATUS: NOT CURRENTLY IMPLEMENTED The features outlined in this section represent conceptual roadmap extensions and are not part of the active production codebase.

1. **Offline PDF Statement Parsing:** Implementing a local, on-device PDF parser capable of digesting password-protected monthly bank and credit card statements, allowing users to backfill years of financial history without SMS.
2. **Encrypted Local Database Export / Import:** Providing an air-gapped export utility that compresses and encrypts `fintrack_secure.db` using a user-specified master passphrase via PBKDF2/AES-256-GCM for secure manual backup to external SD cards or USB drives.
3. **Recurring Subscription & Bill Cadence Detection:** An offline machine-learning engine that scans historical timestamps and merchants to identify recurring monthly charges (e.g. Netflix, Spotify, SIP investments, broadband) and displays proactive payment reminders.
4. **Credit Card Reward & Milestone Optimization:** A rule engine tracking card-specific reward structures (e.g. 5% cashback on Amazon for ICICI Amazon Pay) to advise users on which card to spend with for specific merchants.
5. **Multi-Currency Ledger & FX Tracking:** Incorporating offline currency tables to track foreign transactions and travel expenses across USD, EUR, and GBP.

SECTION 38 — RELEASE & BUILD INFORMATION

38.1 Gradle & Build Configuration

All values below are verified directly against the production repository configuration:

Build Dimension	Repository Configuration
Compile SDK	36 (Android 16 Vanilla Ice Cream+)
Target SDK	36
Minimum SDK	26 (Android 8.0 Oreo)
Kotlin Version	2.3.20
Android Gradle Plugin	9.4.1
Gradle Daemon Version	9.6.0
JVM Toolchain	OpenJDK 17 (<code>JavaVersion.VERSION_17</code>)
Compose Compiler Plugin	Enabled via <code>alias(libs.plugins.compose.compiler)</code>
Kotlin Serialization	Enabled via <code>alias(libs.plugins.kotlin.serialization)</code>
ProGuard / R8	<code>isMinifyEnabled = false</code> (Release build)
Application ID	<code>com.example.fintrack</code>
Version Code	2
Version Name	"2.0"

38.2 Release Signing & Credential Sanitization

FinTrack release builds are cryptographically signed using a dedicated local Java KeyStore.

CAUTION

CREDENTIAL HYGIENE NOTICE: In accordance with strict security standards, all private keystore passwords, store keys, and alias credentials have been sanitized and redacted.

```
# keystore.properties (Sanitized Reference)
storeFile=keystore/fintrack-release.jks
storePassword=[REDACTED]
keyAlias=fintrack
keyPassword=[REDACTED]
```

38.3 Reproducible Build Commands

- **Assemble Release APK:**

```
./gradlew assembleRelease
```

- **Execute Complete Unit Test Suite:**

```
./gradlew test
```

- **Install Upgraded Production APK to Connected Physical Device via ADB:**

```
adb install -r app/build/outputs/apk/release/app-release.apk
```

SECTION 39 — FINAL END-TO-END ARCHITECTURE DIAGRAM



```

- Encrypts: Amounts, Balances, Merchants, Notes
  |
  v
[AppDatabaseHelper]
- Atomic SQLite Write (fintrack_secure.db)
- Inserts Transaction & Updates Balances
- Emits event on _dbChangeSignal SharedFlow
  |
  v
[TransactionRepository]
- Reactive Flow<DashboardSummary>
- Reactive Flow<List<TransactionWithDetails>>
- Reactive Flow<List<Account>>
  |
  v
[MainScreenViewModel]
- Consumes Repository Flows
- Exposes immutable StateFlow<MainUiState>
  |
  v
[Jetpack Compose UI Layer]
- DashboardScreen (Net Worth & Upcoming Dues)
- AccountsScreen (Bank Cards & EMV Credit Cards)
- TransactionsScreen (Categorized Ledger & Badges)
- SmsIntelligenceScreen (Unclassified Alert Reviews)

```

SECTION 40 — FINAL DEVELOPER SUMMARY

40.1 How FinTrack Works in 5 Minutes

FinTrack is an autonomous personal finance intelligence engine designed to turn raw bank SMS alerts into a double-entry-style multi-instrument financial ledger without ever connecting to the internet.

1. **Air-Gapped by Design:** The app has no network permissions. It does not have an API server, does not connect to Plaid, and does not upload logs. It lives entirely inside the local device sandbox.
2. **Multi-Instrument Architecture:** FinTrack explicitly separates **Liquid Assets** (Bank Accounts and Cash Wallets) from **Revolving Liabilities** (Credit Cards). Swiping a credit card increases liabilities and reduces available credit limits, leaving bank accounts untouched. Paying a credit card bill simultaneously reduces bank cash and reduces credit card debt.
3. **Event-Driven Ingestion:** When an SMS arrives from an Indian bank (such as HDFC, SBI, ICICI, Axis, or Kotak), Android wakes `SmsBroadcastReceiver`. Using `goAsync()`, the receiver offloads parsing to a background coroutine on `Dispatchers.IO`.
4. **Deterministic Intelligence:**
 - `IndianBankSmsParser` filters OTPs and extracts entities.
 - `SmsFingerprintEngine` hashes transaction metadata with 2-minute time bucketing to drop duplicates.
 - `AccountIdentificationEngine` maps the SMS to the correct card or bank account using a 4-tier confidence ladder. If ambiguous, it flags `needsReview = true` rather than guessing.
 - `CreditCardPaymentEngine` and `TransferMatchingEngine` correlate dual-instrument movements.

- `BalanceReconciliationEngine` compares the SMS "Available Balance" against local ledger calculations, logging any drift.
5. **Hardware AES-256-GCM Encryption:** All monetary values, merchants, and raw SMS bodies are encrypted using a master key sealed inside the physical device's hardware KeyStore before being written to SQLite (`fintrack_secure.db`).
 6. **Reactive Presentation:** The database notifies `TransactionRepository`, which streams updates through Kotlin Flows to `MainScreenViewModel`. The UI updates instantly via Jetpack Compose Material 3 components.

40.2 Where Should I Start Modifying the Code?

If you are a developer looking to extend or modify FinTrack, here is your roadmap:

To Modify Or Extend...	Inspect and Edit These Specific Files
SMS Parsing & Bank Formats	<code>IndianBankSmsParser.kt</code> and its unit test <code>IndianBankSmsParserTest.kt</code> .
Account Matching & Aliases	<code>AccountIdentificationEngine.kt</code> and <code>AccountAlias.kt</code> .
Credit Card Calculations	<code>Account.kt</code> (see <code>availableCreditLimitCalculated</code>), <code>CreditCardPaymentEngine.kt</code> , and <code>CreditCardDetailsScreen.kt</code> .
Transactions & Transfer Links	<code>Transaction.kt</code> and <code>TransferMatchingEngine.kt</code> .
Database Schema & SQL Queries	<code>AppDatabaseHelper.kt</code> . Remember to increment <code>DATABASE_VERSION</code> and add migrations to <code>onUpgrade()</code> .
Balance Drift & Reconciliation	<code>BalanceReconciliationEngine.kt</code> .
Dashboard & UI Analytics	<code>DashboardScreen.kt</code> and <code>MainScreenViewModel.kt</code> .
Security, Keys & Biometrics	<code>SecurityManager.kt</code> , <code>PinManager.kt</code> , and <code>AuthScreen.kt</code> .
Unit & Integration Tests	<code>app/src/test/java/com/example/fintrack/</code> .

v1 → v2 CHANGE SUMMARY

The following concise log summarizes all engineering modifications delivered in FinTrack v2:

1. Added

- **Domain Models:** `AccountAlias.kt`, `ImportedSmsAlert.kt`, `ReconciliationLog.kt`, `DashboardSummary.kt`.
- **Intelligence Engines:** `AccountIdentificationEngine.kt`, `CreditCardPaymentEngine.kt`, `TransferMatchingEngine.kt`, `BalanceReconciliationEngine.kt`, `SmsFingerprintEngine.kt`.
- **UI Screens & Dialogs:** `CreditCardDetailsScreen.kt`, `SmsIntelligenceScreen.kt`, `PayCreditCardDialog.kt`, `AdjustBalanceDialog.kt`.

- **Security & Biometrics:** `androidx.biometric` integration in `MainActivity.kt` and `AuthScreen.kt`; brute-force lockout in `PinManager.kt`.
- **Test Suites:** `AccountIdentificationEngineTest.kt`, `CreditCardPaymentEngineTest.kt`, `SmsFingerprintEngineTest.kt`.

2. Modified

- **Account.kt** : Added 11 new fields (`accountType`, `bankAccountType`, `creditLimit`, `availableCredit`, `statementDate`, `paymentDueDate`, `minimumDue`, `totalDue`, `linkedPaymentAccountIds`, `lastConfirmedBalance`, `lastConfirmedAt`).
- **Transaction.kt** : Added 8 new fields (`sourceAccountId`, `destinationAccountId`, `kind`, `availableCreditAfterTxn`, `needsReview`, `reviewReason`, `fingerprint`, `linkedTransactionId`).
- **IndianBankSmsParser.kt** : Expanded regex coverage to parse credit card spends, dues, statements, refunds, reversals, ATM withdrawals, and available credit limits across Indian banks.
- **DashboardScreen.kt** : Upgraded to multi-instrument cockpit: Liquid Net Worth, Bank Assets vs Card Liabilities split, and Upcoming Dues banner.
- **AccountsScreen.kt** : Added physical EMV Credit Card renderers with utilization progress bars and quick payment actions.
- **AppDatabaseHelper.kt** : Upgraded to Schema Version 3; added atomic multi-instrument payment mutations and reactive flow change notifications.
- **TransactionRepository.kt** : Integrated complete multi-engine ingestion pipeline.

3. Removed / Deprecated

- Flat single-account assumptions where credit card transactions were dropped or treated as bank debits.
- Unsafe synchronous parsing on the telephony receiver thread.
- Plaintext balance modifications without discrepancy auditing.

4. Migrated

- SQLite Database upgraded from Schema Version 2 to Version 3 via in-place, non-destructive `ALTER TABLE` operations and backfilling of legacy records.

5. Security & Verification

- Field-level hardware AES-256-GCM encryption verified across all sensitive columns.
- Zero network permissions verified (`android.permission.INTERNET` omitted).
- All 21 unit tests passing with 100% success rate.